CS603

Software Architecture and Design

ALI ABBAS



Compiled by: ALI ABBAS

BS SOFTWARE ENGINEERING

Contents

Lesson 1 : 1.1. Design – What and Why 5
Lesson 2 : 1.2. Design – Objectives 7
Lesson 3 : 1.3. Software Design – Complexity 9
Lesson 4 : 1.4. Software Design – Complexity and Size 10
Lesson 5 : 1.5. Types of Complexities 12
Lesson 6 : 1.6. Why is Software Design Hard? 14
Lesson 7 : 1.7. Software Design: A Science Or An Art? 15
Lesson 8 : 1.8. Software Design: A Wicked Problem 17
Lesson 9 : 1.9. Design Process - More Systematic and Predictable 18
Lesson 10 : 1.10. Dealing with Software Complexity: Different Rules and Principles 19
Lesson 11 : 1.11. Characteristics of Software Design 23
Lesson 12 : 1.12. Benefits of Good Design 24
Lesson 13 : 1.13. A Generic Design Process 26
Lesson 14 : 2.14. Inputs to the Design Process 27
Lesson 15 : 2.15. Desirable Internal Design Characteristics 28
Lesson 16 : 2.16. What is good design? 30
Lesson 17 : 2.17. Coupling 31
Lesson 18 : 2.18. Coupling – Design of a Simple Web App 33
Lesson 19 : 2.19. Cohesion 34
Lesson 20 : 2.20. Relationship between Coupling and Cohesion 35
Lesson 21 : 2.21. Software Design Strategies Structured and Object-Oriented Design 36
Lesson 22 : 2.22. Object-Orientation Basic Concepts - The Object 37
Lesson 23 : 2.23. Object-Orientation Basic Concepts - The Class 39
Lesson 24 : 2.24. Relationship between Classes and Objects - Association 40
Lesson 25 : 2.25. Relationship between Classes and Objects: Aggregation and composite 41
Lesson 26 : 2.26. Relationship between Classes and Objects: Inheritance 42
Lesson 27 : 2.27. Object Orientation Basic Concepts Abstract Classes and Interfaces 43
Lesson 28 : 3.28. Relationships – Basic Concepts Object Oriented Programming Defined 44
Lesson 29 : 3.29. Relationships – Basic Concepts Association 45

ALI ABBAS

Lesson 30 : 3.30. Relationships – Basic Concepts Aggregation and Composition 46
Lesson 31 : 3.31. Object Lifetime and Visibility Inheritance 47
Lesson 32 : 3.32. Object Lifetime and Visibility Composition 49
Lesson 33 : 3.33. Object Lifetime and Visibility Association 50
Lesson 34 : 3.34. Object to Relational Mapping Basics 54
Lesson 35 : 3.35. Object to Relational Mapping Mapping Inheritance 55
Lesson 36 : 3.36. Mapping Inheritance - Vertical Mapping 57
Lesson 37 : 3.37. Mapping Composition 58
Lesson 38 : 3.38. Mapping Association 59
Lesson 39 : 3.39. Mapping Cardinality of Association 60
Lesson 40 : 3.40. Object to Relational Mapping – Summary 62
41 - Object Oriented Programming – Inheritance and Polymorphism 63
42 - Object Oriented Programming – Object Creation and Factory Method 65
43 - Object Oriented Programming - The Magic Behind Polymorphism 67
44 - Object Oriented Programming - Implementing Composition 68
45 - Object Oriented Programming - Inheritance vs Composition 71
46 - Implementing Uni-directional Associations 72
47 - Implementing Bi-directional Associations 74
48 - SOLID Design Principles – Introduction 77
49 - SOLID Design Principles - Single Responsibility Principle (SRP) 79
50 - Single Responsibility Principle (SRP) – Example 80
51 - SOLID Design Principles - Open-Closed Principle (OCP) 82
52 - Open-Closed Principle (OCP) – Example 83
53 - Liskov's Substitution Principle (LSP) 85
54 - Liskov's Substitution Principle (LSP) – Example 86
55 - The True Meanings of IsA Relationship 89
56 - SOLID Design Principles - Interface Segregation Principle (ISP) 90
57 - Interface Segregation Principle (ISP) – Example 90
58 - SOLID Design Principles - Dependency Inversion Principle (DIP) 90
59 - Dependency Inversion Principle (DIP) – Example 91
60 - Good OO Design Principles - Law of Demeter (LoD) 91

ALI ABBAS

61 - Law of Demeter (LoD) – Example 91
62 - Action-Oriented Design in the Disguise of Object-Oriented Design 92
63 - Handling Poorly Distributed System Intelligence 92
64 - Modelling Policy – Jacobson's Heuristic 92
65 - Handling Roles - Player Role Pattern 93
66 - Handling Multiple Discriminators Using Player-Role Pattern 94
67 - Software Design and Architecture - Abstraction-Occurrence Pattern 97
68 - Software Design and Architecture - Reflexive Associations 100
69 - Software Design and Architecture - Design Patterns Introduction 102
70 - Software Design and Architecture - Elements of Design Pattern 104
71 - Categories of Design Patterns 106
72 - Benefits and Drawbacks of Design Patterns 108
73 - Singleton Design Pattern 109
74 - Strategy Design Pattern 111
75 - Design Patterns - Introduction - Object-Morphing and Strategy Pattern 112
76 - Facade Design Pattern 117

ALI ABBAS

Lesson 1 : 1.1. Design – What and Why

Overview

In first topic of software architecture and design, we will explore the "Design" meaning in the Cambridge dictionary as verb and noun. We will discuss the application of design in different areas such as design of houses, chairs and helmet. At the end software design and its need will be described in this topic with examples.

Learning Goals

- Learn the design definition as verb and noun
- Learn about the description of design from different areas
- Learn about the software design
- Learn about the need of the design activity

Pre-requisite Courses:

Software engineering I (CS504)

Introduction to programming (CS201)

Pre-existing Knowledge:

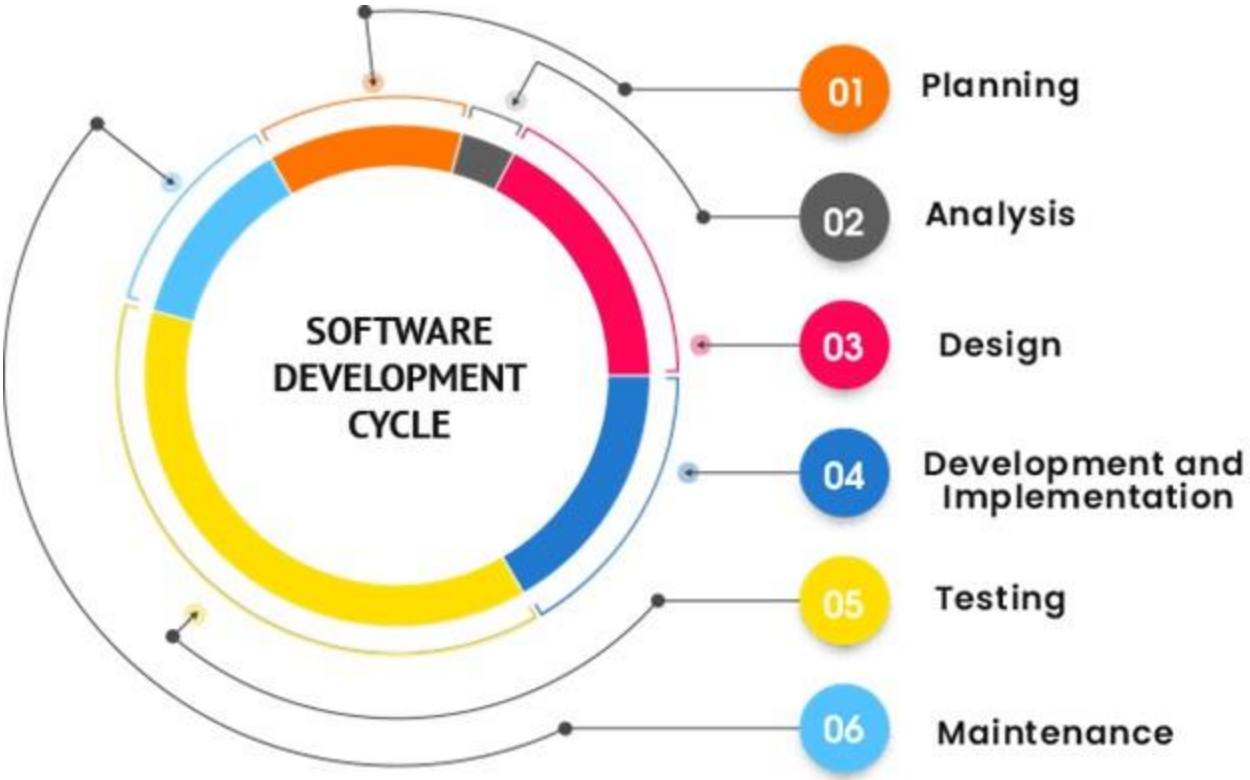
You should know the following concepts prior taking this course

- Basic programming concepts
- Software development life cycle model
- Software requirement specification document (SRS)

Software Application

Software application development process consist on different phases mentioned in the below figure. Planning, analysis, design, deployment and implementation, testing and maintenance are steps (phases) that involved in software development.

ALI ABBAS


SOFTWARE DEVELOPMENT CYCLE
01
Planning
02
Analysis
03
Design
04
Development and Implementation
05
Testing
06
Maintenance

ALI ABBAS

Lesson 2 : 1.2. Design – Objectives

Overview

In this topic, you will get an overview of the design objectives, what we want to achieve from the design, the necessity of the design phase, things involved in design process, what it specifies and how a design should be.

Learning Goals

- Learn about the design objectives
- Learn about the necessity of the design
- Learn about the requirements of the design
- Learn about the problem domain vs design
- Learn about the customer needs and design

ALI ABBAS

Customer Requirements

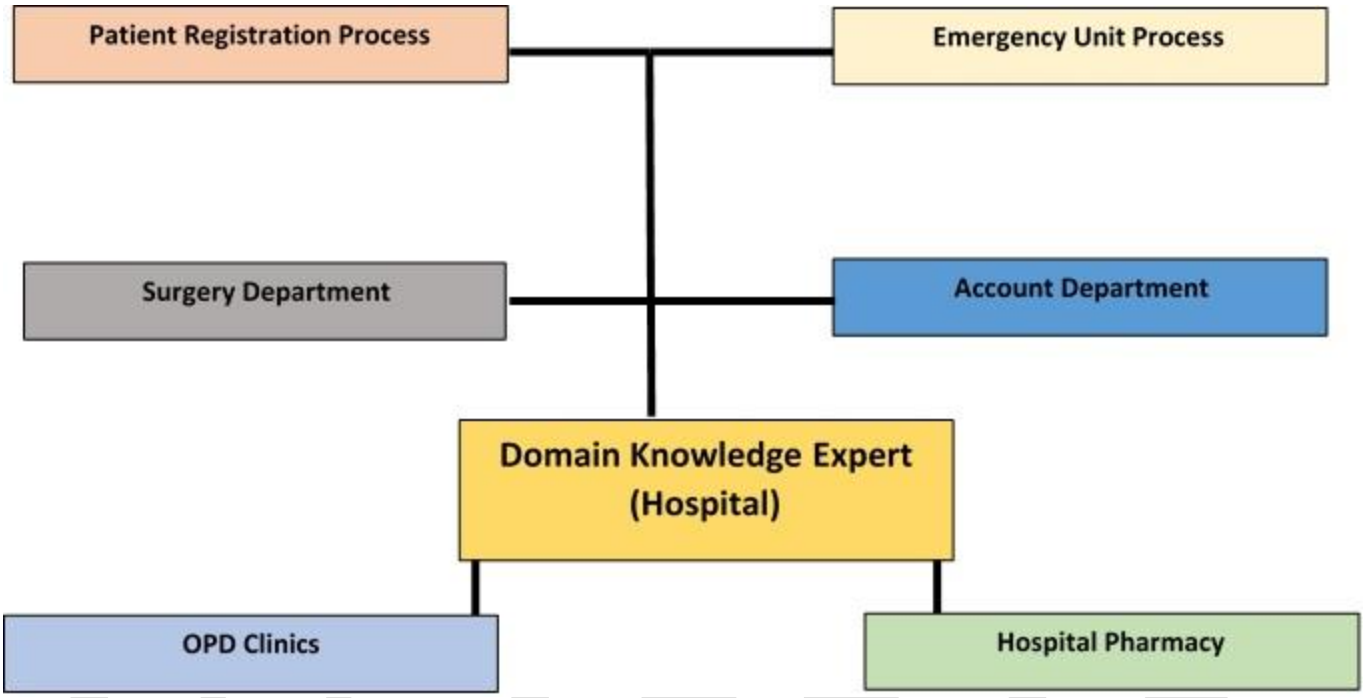
Customer requirements mean what customers or users required from the system. How the system behaves, what are the tasks need to perform by the system, how users will interact with the system, and how the system will present the output. The customer is a significant stakeholder that plays a vital role in the correct development, successful implementation, and system utilization.

Domain Knowledge

Domain knowledge means the information and having expertise about the particular environment and industry in which the developed system will operate. For example, knowledge of the pharmaceutical industry and telecom industry.

Domain Knowledge Expert Example

A person have knowledge of hospitalization process and its major departments.



System Specification

A system specification can be a written document containing a detailed description of all aspects of the system to be built before the project commences.

ALI ABBAS

Lesson 3 : 1.3. Software Design – Complexity

Overview

In this topic, you will get an overview of the design complexity, poorly designed software, why poorly designed software hard to understand and modify. Adding features in poorly and well-designed software effect on cost.

Learning Goals

- Learn about the design complexity
- Learn about the issues with poorly designed software

What is poorly designed software?

Poorly designed means the designer did not spend a good amount of time to analysis software requirements to design. In start, adding small features to show working product to a client is very easy but with the passage of time, software size increased that makes it difficult to understand and make further modifications and adding more features.

ALI ABBAS

What is extendable design?

Change in software requirements and adding more features is very much natural in software development and maintenance process. Well-designed software is easy to understand and welcomes changes in requirements, future modifications and adding new features.

Lesson 4 : 1.4. Software Design – Complexity and Size

Overview

In this topic, you will get to know the complexity of an object with respect to its size. Application of software engineering in software development.

Learning Goals

- Learn about the complexity vs size
- Learn about the application of software engineering
- Learn about the size does matter
- Learn about the consequence of the poor design

Complexity Vs Size (Example of a Building)

The building in the below image is tall with many floors; the building may contain many offices, commercial shops, parking areas, and apartments. Each entity (offices, shops, .etc.) have different kind of facilities, fire exit directions, earthquake resistance, sanitary and sewerage. Due to its large size, any addition or modification will be hard to manage and demand excellent design and architectural skills. In contrast, a single room house is easy to manage and extend.





Complexity VS Size (Example of two different Applications)

A calculator with four functionalities, addition, subtraction, multiplication, and division, is easy to manage, modify, or add any new functionality. While a big application, enterprise resource planning (ERP), contains many different integrated components, it is hard to modify, manage, or add a new feature. Because a calculator has 100 to 200 lines of code, but an ERP solution has millions of lines of code so a single change in ERP may affect other components. You can imagine the complexity relation with the size in the above example; the bigger the size, the bigger the complexity, while the smaller the size, the smaller the complexity.

ALI ABBAS

Lesson 5 : 1.5. Types of Complexities

Overview

In this topic, you will study the types of complexity, reason of these complexities occurrence in the design and development process.

Learning Goals

- Learn about the types of complexities
- Learn about the examples of types of complexities

Types of Complexities

- Essential Complexities
- Accidental complexities

ALI ABBAS

Essential Complexities

You will understand Essential Complexities after reading below different descriptions of essential complexities.

- Essential Complexity is just the nature of the beast you are trying to tame.
- Essential Complexity represents the difficulty inherent in any problem.
- Essential Complexity is Complexity inherent to the problem. It is Complexity related to the problem and cannot remove.
- Essential Complexity is how hard something is to do, regardless of how experienced you are, what tools you use or what new and flashy architecture pattern you used to solve the problem. Essential Complexity is Complexity inherent to the problem.
- Essential Complexity is the entanglement/combination of components/ideas in software necessary for solving the problem at hand. It cannot avoid.

Essential Complexities Examples

Example 1

If users need a program to do 30 things, then those 30 things are essential; you cannot simply take out a few of them to make the software less complex. Whenever you are solving a problem, there are just some areas of complexity that cannot be whittled down.

Example 2

Coordinating a nation's air traffic is an inherently complex problem. Every plane's exact position (including altitude), speed, direction, and destination must be tracked in real-time to prevent mid-air and runway collisions. The flight schedules of aircraft must be managed to avoid airport congestion in a continuously changing environment—a severe change in weather throws the entire schedule out of whack.

Accidental Complexities

You will understand Essential Complexities after reading below different descriptions of essential complexities.

- Accidental Complexity is Complexity not related to the problem. Ben Mosely and Peter Marks describe it as a "mishap." It is Complexity from the fault of the developer and happens to be there.
- Accidental Complexity is the entanglement of components/ideas that is not necessary for solving the problem. This Complexity is accidental because someone probably didn't think hard enough before unnecessarily tying things together. As a result, the software is harder to understand than it should be.
- Accidental Complexity grows from the things we feel we must build to mitigate essential Complexity.

ALI ABBAS

Accidental Complexities Examples

Example 1

Accidental complexity refers to challenges that developers unintentionally make for themselves due to trying to solve a problem. (Fortunately, this kind of complexity can also be fixed or improved by developers.)

Example 2

Developers bring complexity while writing the program. This type of complexity is called accidental complexity.

How to remove accidental complexities?

Accidental complexities can be removed or reduce with the better design.

Lesson 6 : 1.6. Why is Software Design Hard?

Overview

In this topic, you will study, why software designs are hard, creativity in software designs, problems in designs and abstraction.

Learning Goals

- Learn about why design hard
- Learn about design problems
- Learn about the abstraction

Why software design is very important to manage complexity?

One of the most important techniques for managing software complexity is to design systems so that developers only need to face a small fraction of the overall complexity at any given time. Because software designer provides a road map (blueprint) to the developer in which he mentions the system's overall structure.

ALI ABBAS

What is abstraction?

An abstraction is a simplified view of an entity, which omits unimportant details. The software designer ignores the minor details attached to the problem and focused on the high-level details.

For example, a software designer designs the system's overall structure instead of working on the minor details of the application, such as the color of user interface forms and font styles of text.

Why software design is hard?

The fundamental problem is that designers must use current information to predict a future state that will not come about unless their predictions are correct. The outcome of designing has to be assumed before the means of achieving it can be explored: the designers have to work backward in time from an assumed effect upon the world to the beginning of a chain of events that will bring the effect about.

Lesson 7 : 1.7. Software Design: A Science Or An Art?

Overview

In this topic, you will study the relation of software design with science and art. Need of creativity in software designing. Science is fundamental principle and art is creativity.

Learning Goals

- Learn about science
- Learn about art
- Learn about relation of science and art in design

What is Art?

Art is a diverse range of human activities involving creative imagination to express technical proficiency, beauty, emotional power, or conceptual ideas.

ALI ABBAS



What is Science?

- Science is made up of fundamental principles, which can be taught as truth.
- A branch of knowledge or study dealing with a body of facts or truths systematically arranged and showing the operation of general laws.



Lesson 8 : 1.8. Software Design: A Wicked Problem

Overview

In this topic, you will study the wicked problem, meaning of guru and his expertise. Moreover, you will study that the software design problem is a 'wicked' one and this imposes constraints upon the way in which the process of design can be organized and managed.

Learning Goals

- Learn about the wicked problem
- Learn about guru expertise level
- Learn about that design is a wicked one

ALI ABBAS

Lesson 9 : 1.9. Design Process - More Systematic and Predictable

Overview

In this topic, you will understand about the design process, how you will apply it and improve it with the application of methods, techniques, patterns and heuristics.

Learning Goals

- Learn about the design process
- Learn about design process application and improvement

Practice Makes the Man Perfect



ALI ABBAS

How will we understand the design process? A question arises in our minds. How can we apply a design process to make better software design? Design is a creative activity; every person has different skill levels and techniques, some have more, and some have less. However, through continuous learning processes and practice, we can improve our skills and techniques. The design process would be more systematic and predictable by applying methods, techniques, and patterns according to principles and heuristics. After continues learning and practicing, a designer will be able to get good designs. As a designer grows in this profession, he will develop his principles, guidelines, and heuristics. You may not be able to document and express all expertise. In the coming videos, we will learn established principles, procedures, and heuristics by gurus.

Lesson 10 : 1.10. Dealing with Software Complexity: Different Rules and Principles

Overview

In this topic, you will understand the different ways to deal with software complexity. Divide and conquer techniques to slice a problem. You will get to know the modularity, hierarchical organization of the sliced problem, information hiding and abstraction.

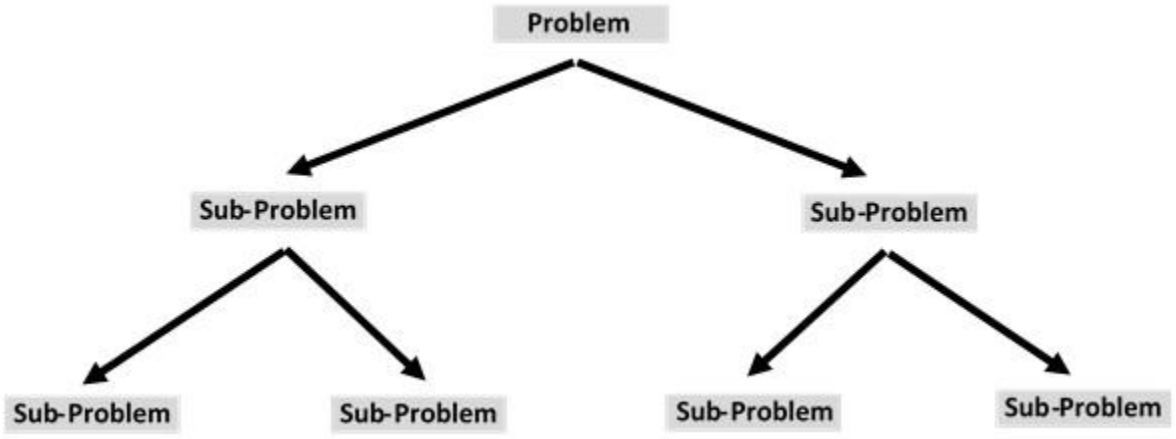
Learning Goals

- Learn about the different ways to deal with complexity
- Learn about modularity
- Learn about the hierarchical organization
- Learn about the information hiding
- Learn about the abstraction

ALI ABBAS

Divide and Conquer

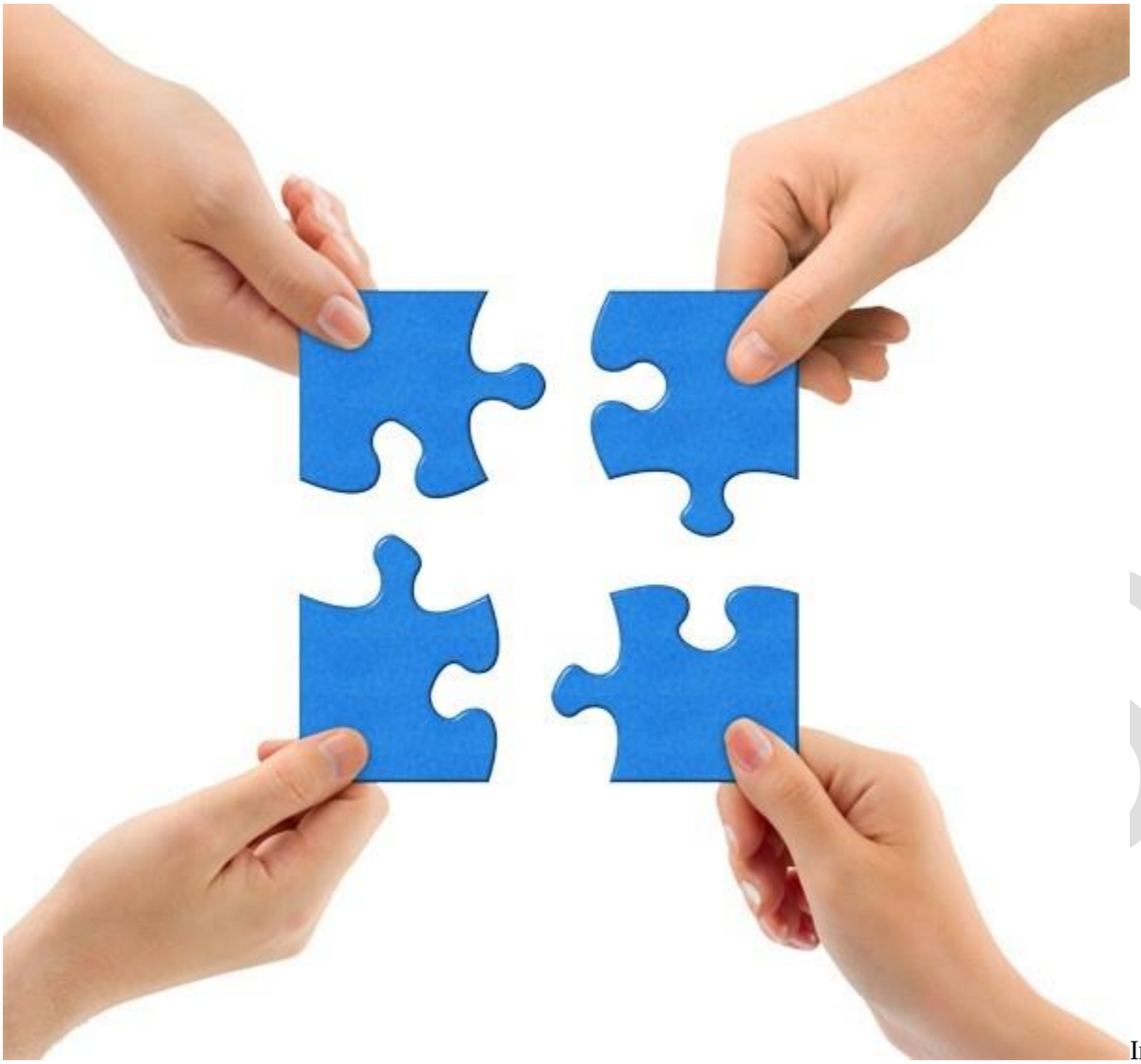
This topic is related to a strategy for solving a problem. If a problem is P and it is a large problem. Suppose its size is n. Then we will divide P into smaller sub-problems P1, P2, P3……. Pk. So a big problem has been divided into smaller sub-problems. Obtaining a solution to each smaller problem is easy rather than solving a big problem at once. All sub-problems are solved individually. After getting the solution to each sub-problem, combine all solutions into one big solution of the one big problem. If the sub-problem is also big then apply the same divide and conquer strategy to divide this sub-problem further.



Modularity

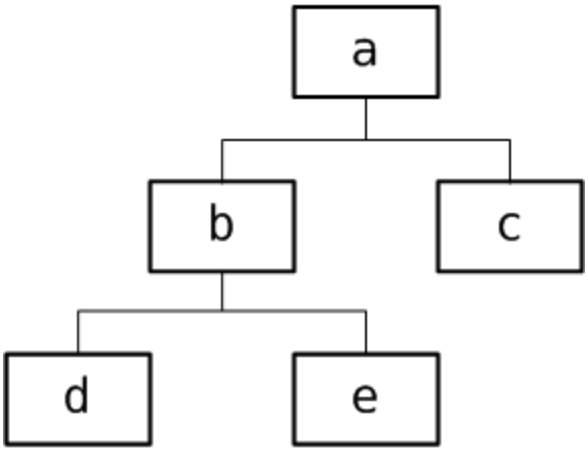
The dictionary defines module as “each of a set of standardized parts or independent units that can be used to construct a more complex structure.”

For example: *'*ships are now built in modules rather than built in a whole from the base up*'*.

 In above figure, you can imagine that four individual modules have been solved individually then combined all modules into one combined solution.

Hierarchical Organization / Structure

A big problem is broken into smaller meaningful parts (Sub-systems) that can be solved individually then we need to combine all these parts that depict the solution of a big problem. But how will we know which part will be integrated with which part? How we will know, which part will be top-level and other parts will come as the sub-part of the top-level part. So Hierarchical Organization provides the solution to organize each part in proper Hierarchy/structure.



Information Hiding

- A component encapsulates its behaviors and data, hiding the implementation details from other components. Modules should be designed so that information (i.e., algorithms and data) contained within one module is inaccessible to other modules that have no need for such information.
- Information hiding, the purpose of information hiding is to separate interface from implementation. By hiding the implementation details that are likely to change, encapsulation allows for flexibility in design. For example, a stack interface can define two public operations, push and pop, that are available to other parts of the program. However, its internal data organization, e.g., whether to use a linked list or an array for storing stack elements, can be hidden.

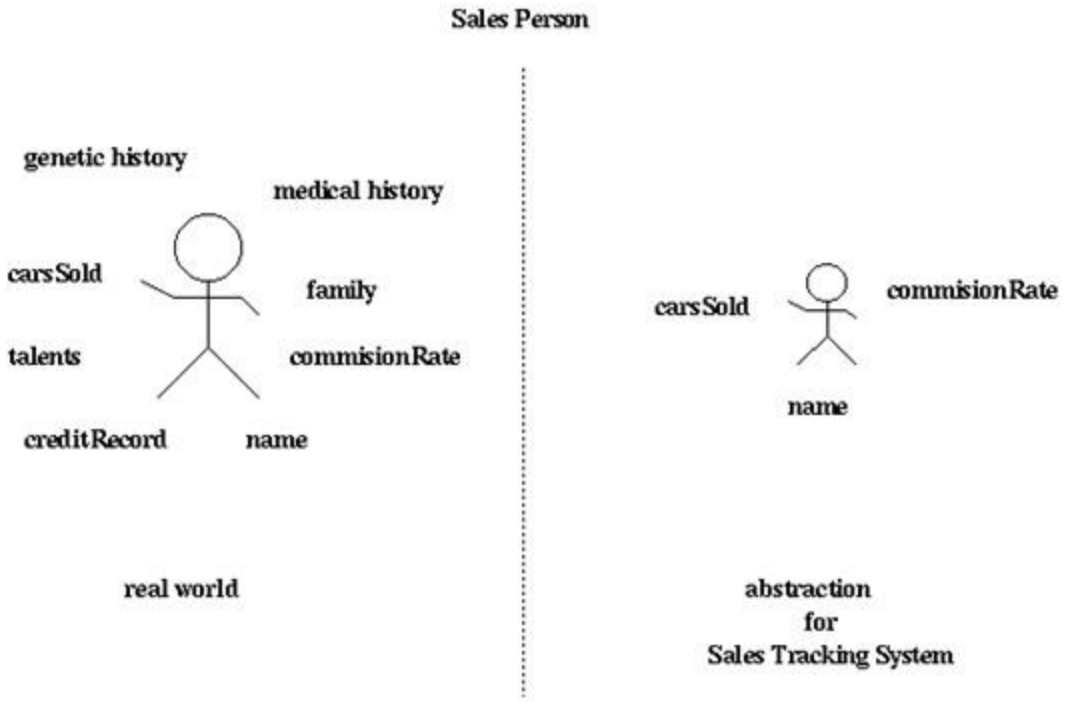
ALI ABBAS

Abstraction

Transitioning your thought process from program design to software design involves the use of abstraction. When designing software, you think of ways to abstract away certain details. This is typically done by grouping certain details together based on shared characteristics or purposes. These shared characteristics or purposes are then a way to generalize your design. For example, a programmer may think about a soccer ball, tennis ball, baseball, and softball as distinct types of items. A software designer would generalize these items by classifying all of them as a type of ball that share certain characteristics (e.g., they are all spheres with a center point and radius) and purposes (e.g., they are all struck by an object). (Reference Book: Guide to Efficient Software Design by David P. Voorhees)

Abstraction Hand-on-Exercise

You are required to explain and discuss below image moderate discussion board.


Sales Person
genetic history
medical history
carsSold
family
talents
commisionRate
creditRecord
name
real world
carsSold
commisionRate
name
abstraction
for
Sales Tracking System

ALI ABBAS

Lesson 11 : 1.11. Characteristics of Software Design

Overview

In this topic, you will learn about the design characteristics, deterministic process, design is non-deterministic, design is heuristic base, design is evolved with the passage of time and design is an emergent process.

Learning Goals

- Learn about the design characteristics

ALI ABBAS

Lesson 12 : 1.12. Benefits of Good Design

Overview

In this topic, benefits of good design is discussed. A good design benefits includes easy to understand, easy to implement and easy to maintain. Furthermore, benefits of code reuse is discussed. How quality will increase with the code reuse and how complexity is the root cause.

Learning Goals

- Learn about the benefits of good design
- Learn about the code reuse and quality
- Learn about the complexity is the root cause

Why code reuse is important?

If you have developed a software, library, class, written an algorithm or written a function. If you have written it well and test properly. Moreover, this functionality is used repeatedly in your software. Its mean that you can call/copy/paste this functionality in your code. In return, your development effort will be less, time will be saved and you can utilize your time efficiently. One more benefit of code reuse is an enhancement in the quality. For example, a code written that tested multiple times and used in different locations in your code will increase the quality of the software and boost the confidence.

ALI ABBAS

Software Quality in regard to Design

The concept of quality is a familiar one, although we tend to associate it with properties that arise from the tasks of construction rather than with those of design. Asked for examples of 'good' quality, many people are likely to identify such examples as well-made joints on an item of furniture, the quality of the paintwork on an expensive car, or the texture of a woven fabric. All these are examples of quality of construction rather than of design, yet quality in construction depends upon good design: to achieve a high-quality product high standards are needed in both, and in order to achieve high standards of quality in design products, one needs to seek high standards of quality in the design process. Unfortunately, quality is a concept that can rarely be related to any absolutes, and even for manufactured items ideas about quality usually cannot be usefully measured on any absolute scale. The best that we can usually do is to rank items on an ordinal scale, as when we say that we believe the construction of this coffee table is better than that of another, in terms of materials used and the workmanship it exhibits. However, we lack any means of defining how much better the one is than the other. When it comes to assessing design, rather than construction, the problem is, if anything, worse. Quality in design is often associated with visual properties, as when we appraise the quality typefaces, furniture and clothes. We associate elegance with good design, yet it offers no help in quantifying our thinking and, even worse, is likely to have ephemeral aspects: it may be strongly influenced by our ideas about fashion – so creating a context for quality. (As an example, a 1930s radio set might look quite in place in a room that is furnished in 1930s style, but would be most unsuited to a room that was furnished in the style of the 1980s.)

No matter how good the design is, in terms of proportions, efficient use of materials and ease of construction and modification, our ideas about its quality will still be very largely influenced by the actual construction. If it is badly assembled, then, regardless of how good its design may be, we will not consider it to be of good quality. Equally, if the design is poor, no amount of good craftsmanship applied to its construction will be able to disguise its fundamental failings. (The door may be well made and may close well, but if it is positioned so that we graze our knuckles each time it is opened, then we will still find it inconvenient to use.)

[Book: Software Design by David Budgen]

Exercise: Analyze the below image regarding code reusability and describe that what comes to your mind

ALI ABBAS

Lesson 13 : 1.13. A Generic Design Process

Overview

In this topic, following areas will be discussed such as, design process, the final design evolves from experience and feedback. Design is an iterative and incremental process, where a complex system arises out of relatively simple interactions. Software design steps, for example, understanding the problem (requirements), construct the black box model of solution (system specification), look for existing solutions (e.g. architecture and design patterns) that cover some or all of the software design problems identified. Discover missing requirements, consider building prototypes, document and review design, iterate over solution (refactor) (evolve the design until it meets functional requirements and maximize non-functional requirements).

Learning Goals

- Learn about the design process
- Learn about the design is iterative and incremental process
- Learn about the steps involved in design process

ALI ABBAS

Lesson 14 : 2.14. Inputs to the Design Process

Overview

In this topic, you will study the different inputs of the design process, for example, user requirements and system specification including constraints on design and implementation. You will study the Importance of domain knowledge.

Learning Goals

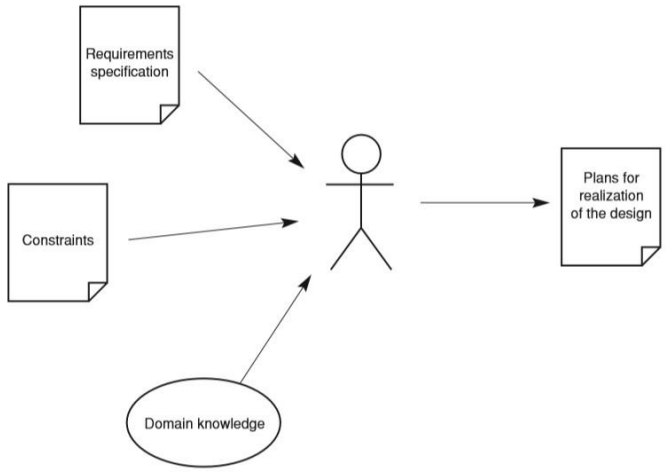
- Learn about the inputs of the design process
- Learn about the importance of domain knowledge

Constraint

A constraint is a decision over which you have little or no control as a designer/ architect. Your job is just to satisfy to design the best system that you can, despite the constraints you face. Sometimes you might be able to argue for loosening a constraint, but in most cases, you have no choice but to design around the constraints.

Designer's Communication Channels

A software designer may need to acquire some degree of 'domain knowledge' as a routine part of the input needed for undertaking any particular design task. Below figure present, the different communication channels (Inputs to the design process) through which the designer acquire the information needed in the design process.



Lesson 15 : 2.15. Desirable Internal Design Characteristics

Overview

In this topic, you will study; desirable design internal characteristics, poorly designed programs difficult to understand and hard to modify, ease of maintenance one of the prime objective, code will be read more often than it is written, minimal complexity, keep it simple, system should be extendable, engineering is all about balancing conflicting objectives, loose coupling, reusability and portability.

Learning Goals

- Learn about the desirable internal design characteristics
- Learn about ease of maintenance, minimal complexity and portability

ALI ABBAS

Maintainability

As systems get larger and more costly, the need to offset this by ensuring a long lifetime in service increases in parallel. In order to achieve this, designs must allow for future modification.

Simplicity

A characteristic of almost all good designs, in whatever sphere of activity they are produced, is a basic simplicity. A good design meets its objectives and has no additional embellishments that detract from its main purpose.

Reusability Phenomenon

The concept of reuse makes an explicit appearance in the definition provided by Brown and Short, and implicit appearances in the others. Strictly, of course, reuse is not an essential characteristic for a component, only a desirable one. If a large system contains one or two uniquely crafted components among a set of reused ones, then this can be considered as a quite pragmatic design decision that reduces the detailed design problem to that of designing only a few components, rather than a whole system

Jones says,

"*Most experienced programmers have private libraries which allow them to develop software with about 30% reused code by volume. Reusability at the corporate level aims for 75% reused code by volume, and requires special library and administrative support. Corporate reusable code also implies changes in project accounting and measurement practices to give credit for reusability*"

Van Snyder points out,

"*We conjecture that barriers to reuse are not on the producer side, but on the consumer side. If a software engineer, a potential consumer of standardized software components, perceives it to be more expensive to find a component that meets his need, and so verify, than to write one anew, a new, duplicative component will be written. Notice we said perceives above. It doesn't matter what the true cost of reconstruction is. Reuse has been successful for mathematical software for two reasons: (I) It is arcane, requiring an enormous intellectual input per line of code; and (2) there is a rich and standard nomenclature, namely mathematics, to describe the functionality of each component. Thus, the cost to reconstruct a component of mathematical software is high, and the cost to discover the functionality of an existing component is low. The long tradition of professional journals publishing and collecting algorithms, and offering them at modest cost, and commercial concerns offering very high quality algorithms at somewhat higher but still modest cost, makes discovering a component that meets one's need simpler than in many other disciplines, where it is sometimes not possible to specify one's need precisely and tersely. These factors collaborate to make it more attractive to reuse rather than to reinvent mathematical software."*

Parnas writes,

"Reuse is something that is far easier to say than to do. Doing it requires both good design and very good documentation. Even when we see good design, which is still infrequently, we won't see the components reused without good documentation."

[Book: The Mythical Man-Month]

Portability

The system's level of independence on software and hardware platforms. Systems developed using high-level programming languages usually have good portability. One typical example is Java — most Java programs need only be compiled once and can run everywhere.

[Book: Software Architecture and Design Illuminated]

ALI ABBAS

Lesson 16 : 2.16. What is good design?

Overview

In this topic, you will study; design must fulfill user's requirements, comparison of different designs, assessment features of designs, an important feature of assessment design for example, maintainable design, and understandability of designs.

Learning Goals

- Learn about the features of a good design

What are Characteristics of a Good Software Design?

Six characteristics of good software design: simplicity, coupling, cohesion, information hiding, performance, and security

ALI ABBAS

Lesson 17 : 2.17. Coupling

Overview

In this topic, you will study; coupling that is the measure of dependency between modules and components. It is the degree of interdependence between two modules and components. A dependency exists between two modules/components if a change in one could require a change in the other. Coupling is inter-component relationship. How to measure coupling and complexity of each interface? Why high degree coupling is bad, because highly coupled systems have strong interconnection. These systems are difficult to test, reuse and understand separately. Loosely coupled modules are less dependent upon each other and hence are easier to change you can say, low coupling can be achieved by eliminating unnecessary relationship and reducing the number of necessary relationships.

ALI ABBAS

Learning Goals

- Learn about the coupling
- Learn about measuring the coupling
- Learn about difficulties in coupled systems
- Learn about reducing the coupling

Key to Understand the Coupling

The key to understanding coupling is the word module found in the software design usage description. Unfortunately, the word module has been used to describe a wide range of physical objects (e.g., the Apollo Lunar Module, modular home, power supply module) and software constructs (e.g., component, package, class, function).

Module For an object-oriented software design:

- A module is typically a class (i.e., a group of logically related attributes and methods). For example, a class that contains methods that validate data entered by a user. For very large software systems, a module may be a package of classes.
- A less commonly used interpretation is that a module is a single method found in a class definition.

Module For a structured software design:

- A module is typically a group of logically related functions. For example, a set of functions that perform validation of data entered by a user could be considered a distinct module within a structured design. This type of module is also known as a component (or in a very large software system a subcomponent). For example, a Python source code file is called a module. Thus, each Python module should contain logically related functions.

- A less commonly used interpretation is that a module is a single function found in a source code file.

Coupling

The coupling definitions indicate that coupling describes the amount of connectedness between two or more modules. Thus, a design that has only one module cannot exhibit coupling. That is, coupling can only be described between distinct modules.

Our intuition suggests having fewer connections between modules mean there is less to test, maintain, and fix. Having fewer connections between modules is called low coupling, loose coupling, or weak coupling. A good design is one that exhibits low coupling between its modules. In contrast, more connections between modules means there is more to test, maintain, and fix. Having more connections between modules is called high coupling, tight coupling, or strong coupling. A bad design is one that exhibits high coupling between its modules

[Book: Guide to Efficient Software Design by David P. Voorhees, Publisher: Springer; 1st ed. 2020 edition]

ALI ABBAS

Lesson 18 : 2.18. Coupling – Design of a Simple Web App

Overview

In this topic, you will study; coupling relevancy with maintainability or modification. Reflection of coupling in code by web application example.

Learning Goals

- Learn about coupling by code example

ALI ABBAS

Lesson 19 : 2.19. Cohesion

Overview

In this topic, you will study about cohesion. Cohesion refers to the degree to which the elements inside a module belong together. It is a measure of the strength of the relationship between the methods and data of a class and some purpose or concept served by that class. A module has high cohesion if all of its elements are working towards the same goal.

ALI ABBAS

Learning Goals

- Learn about basics of cohesion

Lesson 20 : 2.20. Relationship between Coupling and Cohesion

Overview

Coupling and cohesion are important measures regarding maintainability of the system. Low coupling often relates with high cohesion, and vice versa. Low coupling is a sign of a well-structured software system and when combined with high cohesion, supports the general goals of high readability and maintainability

ALI ABBAS

Learning Goals

- Learn about basics of cohesion

Lesson 21 : 2.21. Software Design Strategies Structured and Object-Oriented Design

Overview

Software design strategies will discuss in this topic. For example, structured design and Object Oriented Design.

ALI ABBAS

Learning Goals

- Learn about structured design
- Learn about object oriented design

Lesson 22 : 2.22. Object-Orientation Basic Concepts - The Object

Overview

You will study about the object, its state, behavior and identity.

Learning Goals

- Learn about the object

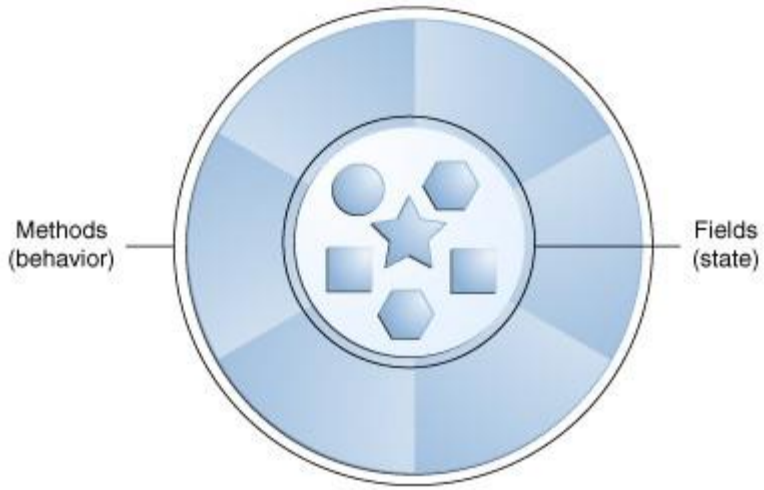
ALI ABBAS

Object

Objects are key to understanding *object-oriented* technology. Look around right now and you'll find many examples of real-world objects: your dog, your desk, your television set, your bicycle.

Real-world objects share two characteristics: They all have *state* and *behavior*. Dogs have state (name, color, and breed, hungry) and behavior (barking, fetching, wagging tail). Bicycles also have state (current gear, current pedal cadence, and current speed) and behavior (changing gear, changing pedal cadence, applying brakes). Identifying the state and behavior for real-world objects is a great way to begin thinking in terms of object-oriented programming.

Take a minute right now to observe the real-world objects that are in your immediate area. For each object that you see, ask yourself two questions: "What possible states can this object be in?" and "What possible behavior can this object perform?". Make sure to write down your observations. As you do, you'll notice that real-world objects vary in complexity; your desktop lamp may have only two possible states (on and off) and two possible behaviors (turn on, turn off), but your desktop radio might have additional states (on, off, current volume, current station) and behavior (turn on, turn off, increase volume, decrease volume, seek, scan, and tune). You may also notice that some objects, in turn, will also contain other objects. These real-world observations all translate into the world of object-oriented programming.



A Software Object

For further reading: [https://docs.oracle.com/javase/tutorial/java/concepts/object.html]

ALI ABBAS

Lesson 23 : 2.23. Object-Orientation Basic Concepts - The Class

Overview

In this topic, you will study about the class. Class represent similar objects in object orientation. Class is a template of an object that includes properties and behavior.

Learning Goals

- Learn about the class in object orientation

The Class

In the real world, you will often find many individual objects all of the same kind. There may be thousands of other bicycles in existence, all of the same make and model. Each bicycle was built from the same set of blueprints and therefore contains the same components. In object-oriented terms, we say that your bicycle is an *instance* of the *class of objects* known as bicycles. A *class* is the blueprint from which individual objects are created.

[For further reading: https://docs.oracle.com/javase/tutorial/java/concepts/class.html]

ALI ABBAS

Lesson 24 : 2.24. Relationship between Classes and Objects - Association

Overview

In this topic, you will study the relationship between classes, association and attributes of association.

Learning Goals

- Learn about the association

ALI ABBAS

Lesson 25 : 2.25. Relationship between Classes and Objects: Aggregation and composite

Overview

In this topic, you will study the aggregation and composite aggregation also known as composition.

Learning Goals

- Learn about the aggregation and composition

ALI ABBAS

Lesson 26 : 2.26. Relationship between Classes and Objects: Inheritance

Overview

In this topic, you will study about classification, inheritance and inheritance hierarchy.

Learning Goals

- Learn about the classification and inheritance

ALI ABBAS

Lesson 27 : 2.27. Object Orientation Basic Concepts Abstract Classes and Interfaces

Overview

In this topic, you will study about the abstract classes and interfaces. Basic concepts of abstract classes and need of the abstract classes. Difference between abstract class and interface.

Learning Goals

- Learn about the abstract classes
- Learn about the interfaces

ALI ABBAS

Lesson 28 : 3.28. Relationships – Basic Concepts Object Oriented Programming Defined

Overview

You will study about the relationship in object orientation Concepts

Learning Goals

- Learn about the relationship in object orientation Concepts

Relationships in Object Oriented Concepts

ALI ABBAS

If you are building a house, things like walls, doors, windows, cabinets, and lights will form part of your vocabulary. None of these things stands alone, however. Walls connect to other walls. Doors and windows are placed in walls to form openings for people and for light. Cabinets and lights are physically attached to walls and ceilings. You group walls, doors, windows, cabinets, and lights together to form higher-level things, such as rooms.

Not only will you find structural relationships among these things, you'll find other kinds of relationships, as well. For example, your house certainly has windows, but there are probably many kinds of windows. You might have large bay windows that don't open, as well as small kitchen windows that do. Some of your windows might open up and down; others, like patio windows, will slide left and right. Some windows have a single pane of glass; others have double. No matter their differences, there is some essential "window-ness" about each of them: Each is an opening in a wall, and each is designed to let in light, air, and sometimes, people. In the UML, the ways that things can connect to one another, either logically or physically, are modeled as relationships.

In object oriented modeling, there are two kinds of relationships that are most important: inheritance and associations.

[Book: Unified Modeling Language (UML) User Guide By Grady Booch James Rumbaugh Ivar Jacobson]

Lesson 29 : 3.29. Relationships – Basic Concepts Association

Overview

You will study about the association and types of association. In type of association, uses relationship and whole-part relationships will discuss.

Learning Goals

- Learn about the association
- Learn about the types of association

ALI ABBAS

Lesson 30 : 3.30. Relationships – Basic Concepts Aggregation and Composition

Overview

You will study about the association's categories, aggregation and composition. Both are whole-part relationships.

Learning Goals

- Learn about aggregation
- Learn about composition

ALI ABBAS

Lesson 31 : 3.31. Object Lifetime and Visibility Inheritance

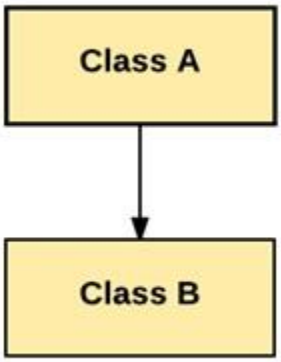
Overview

You will study about the object lifetime and visibility in inheritance. While creating a parent child relationship, which class's object will control the other class's object.

Learning Goals

- Learn about the object lifetime and visibility in inheritance

Lifetime of Objects in Inheritance

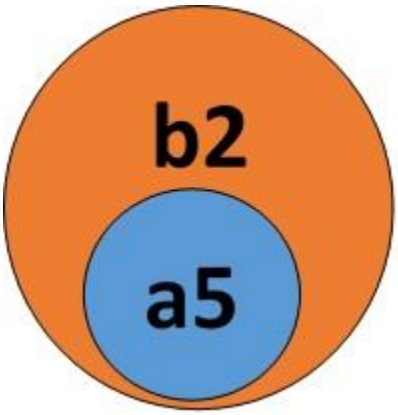


ABBAS

In above inheritance relationship, B inherits from A. So A is the superclass and B is the subclass. While designing the parent-child relationship, parent class does not know about its child class, so it means, A does not know about B. This is the first principle while designing the inheritance.

Secondly, when a child class's object (B) is created, a corresponding object of superclass (A) is also created. Conceptually speaking, the object of Type B holds the corresponding object of Type A as a private component. Here, private component means, A's object is hidden in B's object. You can say, its component of B. A's object is only accessible through class's B object.

For example, when an object, say b2, of Type B is created, another object, say a5, of Type A is also created.



a5 is only accessible from b2. No other object has reference of a5 object. This is called exclusive holding. The life of a5 is dependent upon b2. a5 is created with b2 and it dies with b2. Cardinality of this relationship is 1:1. It means life of a5 is depended upon b2.

ALI ABBAS

Lesson 32 : 3.32. Object Lifetime and Visibility Composition

Overview

In this topic, you will study the lifetime and visibility of objects in composition. An example is given in the video. You must watch the video for clear understanding of this concept.

Learning Goals

- Learn about the lifetime of objects in composition

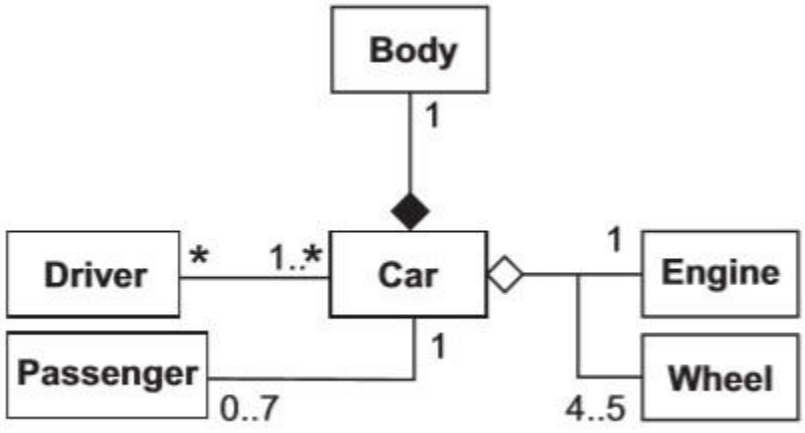
Multiplicity in Composition

For composition, the multiplicity at the composer's end is always 1 because, according to the UML rules, a composed object can't be shared among composites.

ALI ABBAS

For example:

- A Car has one Engine.
- An Engine is part of one Car.
- A Car has four or five Wheels.
- Each Wheel is part of one Car.
- A Car is always composed of one Body.
- A Body is always part of one Car and it dies with that Car.
- A Car can have any number of Drivers.
- A Driver can drive at least one Car.
- A Car has up to seven Passengers at a time.
- A Passenger is only in one Car at a time.



[Book: Object-Oriented Analysis and Design by Mike O'Docherty]

ALI ABBAS

Lesson 33 : 3.33. Object Lifetime and Visibility Association

Overview

In this topic, you will study the association. Association is a direct or indirect connection between two objects. You will study lifetime and visibility of objects.

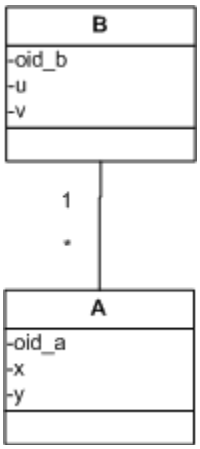
Learning Goals

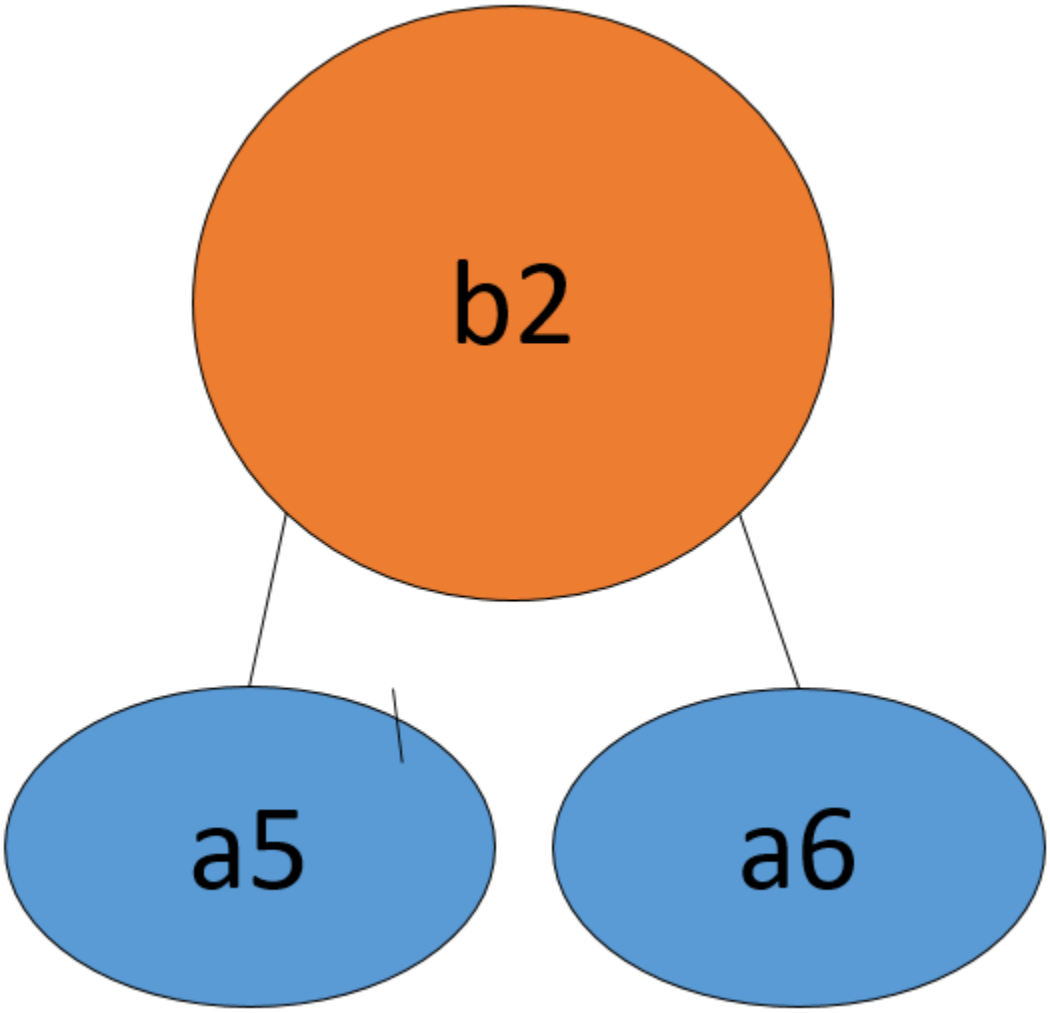
- Learn about the lifetime of objects in association

Object Lifetime and Visibility Association

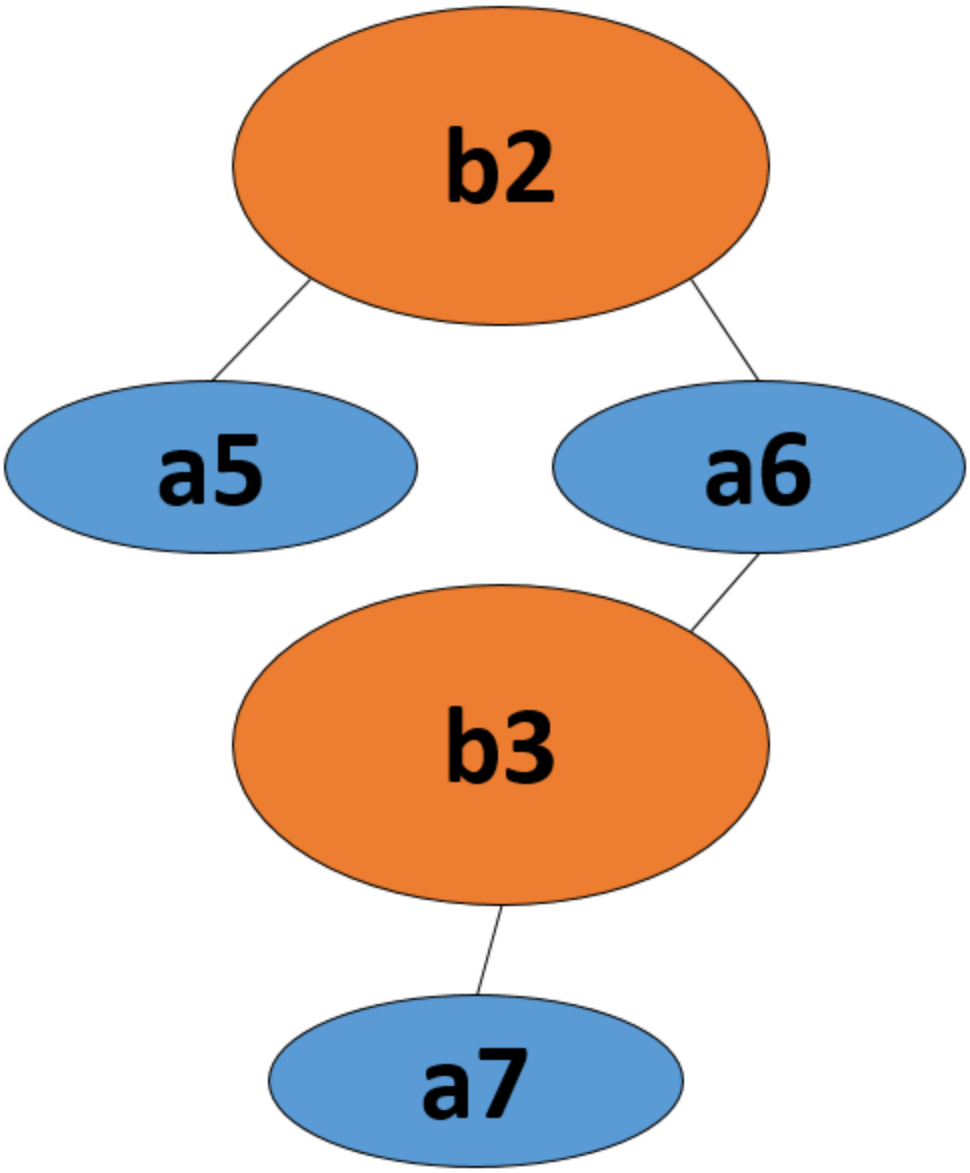
- One instance of B is associated with many instances of A
- Objects of the two types have independent life times.
- Conceptually speaking, objects of Type B are linked with objects of Type A.
- The relationship is not exclusive. That is, the same instance of A may also be linked and accessed directly by other objects.
- Cardinality of this relationship could be 1:1, 1:m, or m:m

ALI ABBAS





Activate Windows


b2
a5
a6
b3
a7

Lesson 34 : 3.34. Object to Relational Mapping Basics

Overview

In this topic, you will study the basic to transform object's data to relational databases using object relational mapper.

Learning Goals

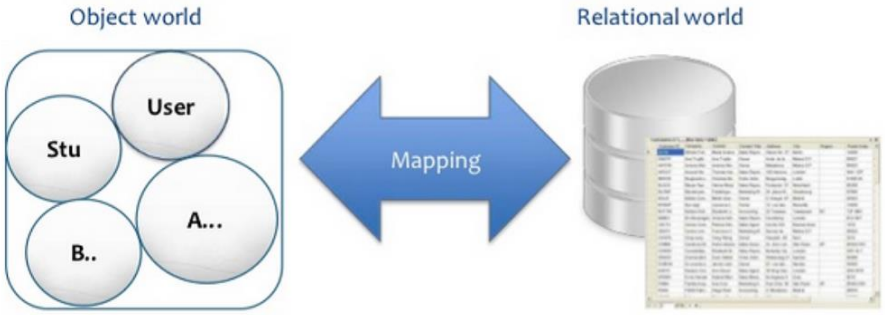
- Learn about the basics of database
- Learn about the object relational mappers

Object to Relational Mapping Basics

- Relational Database
- Objects
- Mapping between Objects & Relational Database

O2R Mapping

- aka – **O2R**, **ORM**, **O/RM**, and **O/R**
- Mapping attributes
- Mapping Inheritance
- Mapping Association
- Mapping Composition



ALI ABBAS

Lesson 35 : 3.35. Object to Relational Mapping Mapping Inheritance

Overview

You will study about the mapping of inheritance in rational database. There are three way to map inheritance into rational model. For example, filtered mapping, horizontal mapping and vertical mapping.

Learning Goals

- Learn about the inheritance mapping into rational model

Object to Relational Mapping: Mapping Inheritance

ALI ABBAS

Mapping Inheritance

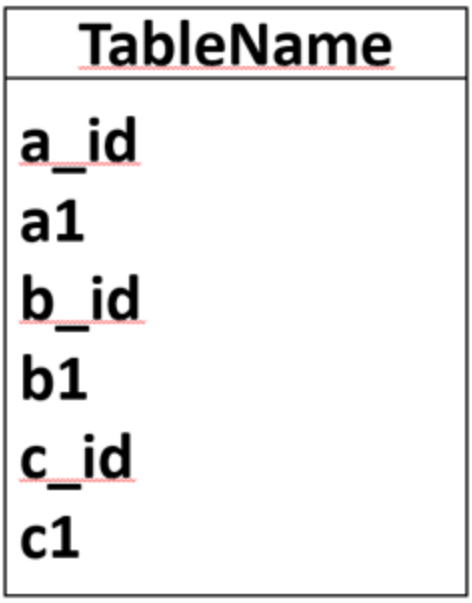
- Filtered Mapping
- Horizontal Mapping
- Vertical Mapping

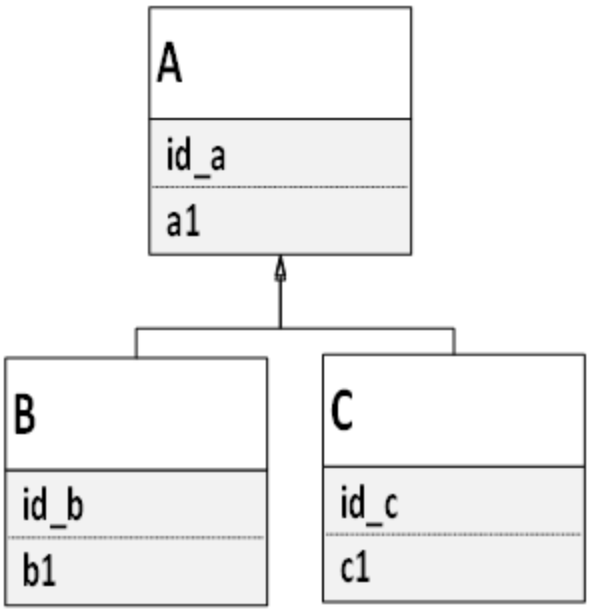
Filtered Mapping

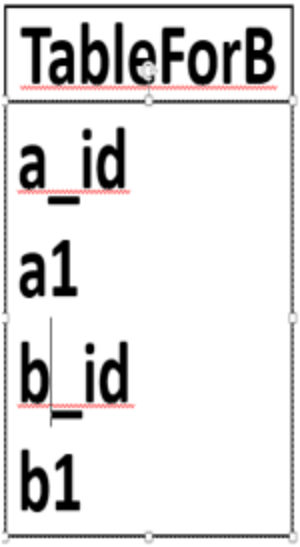
- All the classes of an inheritance hierarchy are mapped to one table.

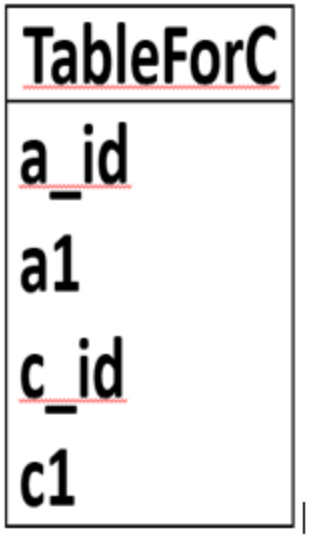
Horizontal Mapping

- Each concrete class is mapped to a separate table and each such table includes the attributes and the inherited attributes of the class that it is representing.


TableName
a_id
a1
b_id
b1
c_id
c1


A
id_a
a1
B
id_b
b1
C
id_c
c1


TableForB
a_id
a1
b_id
b1


TableForC
a_id
a1
c_id
c1

Lesson 36 : 3.36. Mapping Inheritance - Vertical Mapping

Overview

You will study about the mapping of inheritance in rational database. There are three ways to map inheritance into rational model. For example, filtered mapping, horizontal mapping and vertical mapping. In this topic, vertical mapping will be discussed.

Learning Goals

- Learn about the vertical mapping

Vertical Mapping

ALI ABBAS

In this technique, there exists one database table for each class. Separate tables provide stable data storage with constrained definitions, but it is more complex to query a child-class. It requires to write some join statements which reduces the performance. For example, to get a `Customer` object, it needs to be joined with `User` table.

User Table:

UID	Username	Password	Type
U1	Ali	12345	Customer
U2	Aleena	abcde	Maintainer

Customer Table:

ID	CustomerNumber	UID
1	1001	U1

Maintainer Table:

ID	EmployeeNumber	UID
2	1001	U2

Lesson 37 : 3.37. Mapping Composition

Overview

In this topic, you will study the composition mapping into rational table. How whole and part classes are mapped in rational tables.

Learning Goals

- Learn about the composition mapping in rational table

Object to Relational Mapping: Mapping Composition

Vertical Mapping

- Each class of the whole-part hierarchy is mapped to a separate table.
- To maintain the whole-part relationship between the whole (composite class) and the part (component class), primary key (OID) of the composite class is inserted in the part classes as a foreign key.
  - Note the difference between composition and inheritance

ALI ABBAS

Composition – Persistence - Example

- For example, object of type B with oid_b = 21 contains objects of type A with oid_a = 1 and oid_a = 2

Lesson 38 : 3.38. Mapping Association

Overview

In this topic, you will study the association mapping into rational table. Each class involved in the relationship is mapped to a separate table.

Learning Goals

- Learn about the association mapping in rational table

Object to Relational Mapping: Mapping Association

- Each class involved in the relationship is mapped to a separate table.
- To maintain the relationship between the classes, primary key (OID) of the user class is inserted in the classes that have been used as a foreign key.
- Note that, in the OO world, the client knows the identity of the server whereas in the relational world, the table for server holds the identity of the client.
- For many-t-many relationship a separate table is used to maintain the information about the relationship.

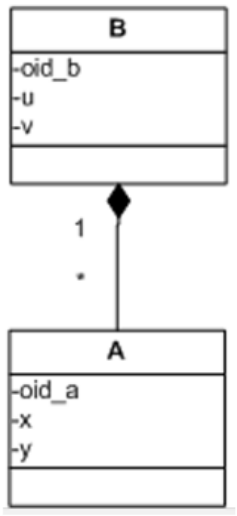
ALI ABBAS

Composition – Persistence - Example

- For example, object of type B with oid_b = 21 uses objects of type A with oid_a = 1 and oid_a = 2

oid_b	u	v
10	23	10
21	34	54
35	78	17

oid_a	oid_b(FK)	x	y
1	21	567	100
2	21	98	43
3	35	718	127



Lesson 39 : 3.39. Mapping Cardinality of Association

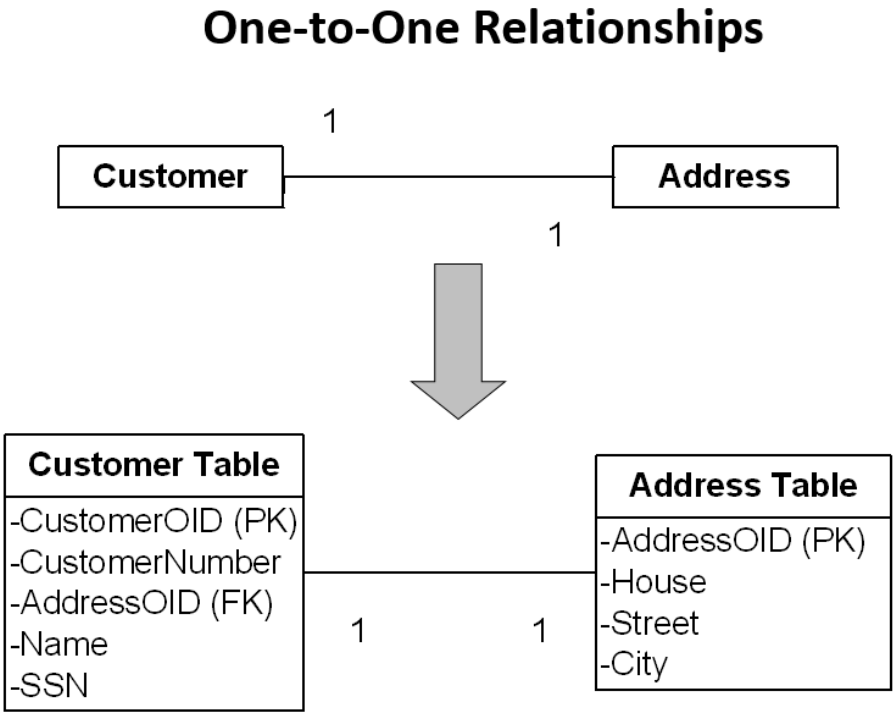
Overview

In this topic, you will study mapping cardinality of association into rational table. Mapping, one to one, one to many and many to many relationship.

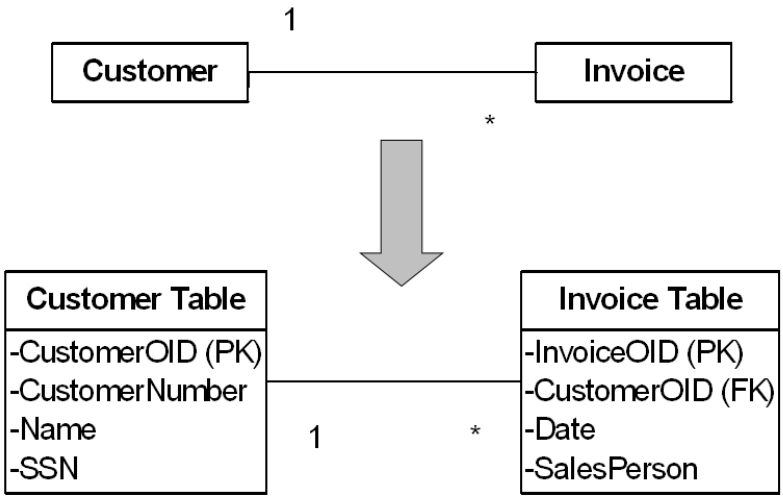
Learning Goals

- Learn about the mapping cardinality of association.

Object to Relational Mapping: Mapping Cardinality of Association

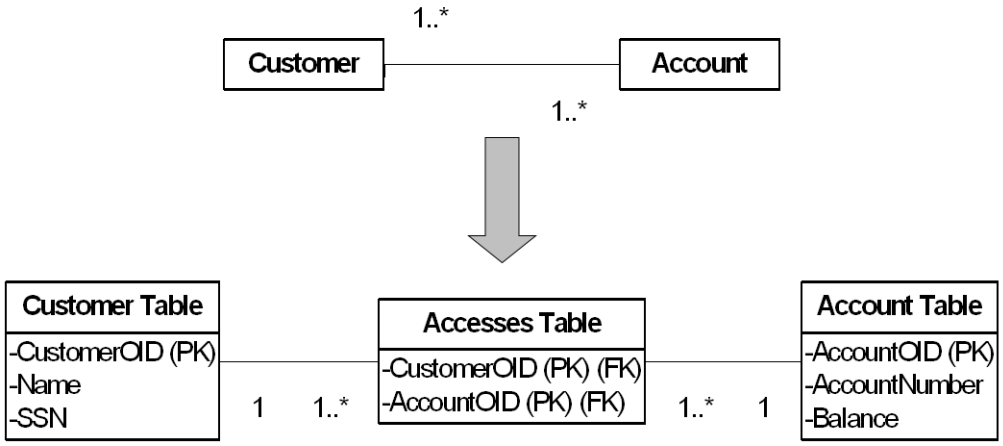


One-to-Many Relationships



ALI ABBAS

Many-to-Many Relationships



Lesson 40 : 3.40. Object to Relational Mapping – Summary

Overview

In this topic, you will study object to rational mapping summary. All relationships, types of relationships, mapping approaches.

Learning Goals

- Summary

Object to Relational Mapping: Summary

	What	Life-time	Holding / visibility	cardinality
Inheritance	Is-a relationship parent-child relationship	Object of base class is created and destroyed with derived class object	Exclusive Parent does not know about the child	1:1
Composition	whole-part relationship	Part is created and destroyed with the whole	Exclusive Part does not know about the whole	1:1, 1:m
Association	Using relationship link	independent	Non-exclusive Can be one-way or bi-directional	1:1, 1:m, m:m

41 - Object Oriented Programming – Inheritance and Polymorphism

Overview

In this topic, you will study polymorphic behavior with example.

Learning Goals

- Polymorphism and inheritance behavior

OOP – Inheritance and Polymorphism

```
class Polygon {
  protected:
    int width, height;
  public:
    Polygon (int a, int b) : width(a), height(b) {}
    virtual int area (void) =0;
    void printarea()
      { cout << this->area() << '\n'; }
};
class Rectangle: public Polygon {
  public:
    Rectangle(int a,int b) : Polygon(a,b) {}
    int area()
      { return width*height; }
};
class Triangle: public Polygon {
```

ALI ABBAS

```
  public:
    Triangle(int a,int b) : Polygon(a,b) {}
    int area()
      { return width*height/2; }
};
int main () {
  Polygon * ppoly1 = new Rectangle (4,5);
  Polygon * ppoly2 = new Triangle (4,5);
  ppoly1->printarea();
  ppoly2->printarea();
  delete ppoly1;
  delete ppoly2;
  return 0;
}
```

ALI ABBAS

Questions

- What do we gain by using the base class in the declaration

  Polygon * ppoly1 = new Rectangle (4,5);

  Polygon * ppoly2 = new Triangle (4,5);

- Why not

  Rectangle * ppoly1 = new Rectangle (4,5);

  Triangle * ppoly2 = new Triangle (4,5);

```
int main () {
        Polygon * ppoly[10];
```

```
        ppoly[0] = new Rectangle (4,5);
        ppoly[1] = new Triangle (4,5);
        // and so on
        for (int i = 0; i < 10; i++)
                        ppoly[i]->printarea();
        // rest of the code
        return 0;
}
```

42 - Object Oriented Programming – Object Creation and Factory Method

ALI ABBAS

Overview

In this topic, you will study object orientation and factory method.

Learning Goals

- Learn object orientation and factory method

OOP – Object Creation and Factory Method

```
int main () {
        Polygon * ppoly[10];
        ppoly[0] = new Rectangle (4,5);
        ppoly[1] = new Triangle (4,5);
        // and so on
        for (int i = 0; i < 10; i++)
                ppoly[i]->printarea();
```

```
        // rest of the code
        return 0;
}
```

Object creation and Factory Method

```
Polygon * createPolygon(char t, int w, int h) {
            if (t == 'R')
                return new Rectangle(w, h);
            else if (t == 'T')
                return new Triangle(w, h);
}
int main () {
        Polygon * ppoly[10];
        for (int i = 0; i < 10; i++) {
            // get type, width, and height of the object
            // to be created
            ppoly[i] = createPolygon(t, w, h);
            // pseudo polymorphic behavior
        }
        for (int i = 0; i < 10; i++)
            ppoly[i]->printarea();
        // rest of the code
        return 0;
}
```

ALI ABBAS

43 - Object Oriented Programming - The Magic Behind Polymorphism

Overview

In this topic, you will study about the magic behind the polymorphism. Detail of vtable and vptr that is very important feature to attain the polymorphic behavior.

Learning Goals

- Learn about the magic behind the polymorphism

OOP – The Magic behind Polymorphism

ALI ABBAS

- Compiler maintains two things:

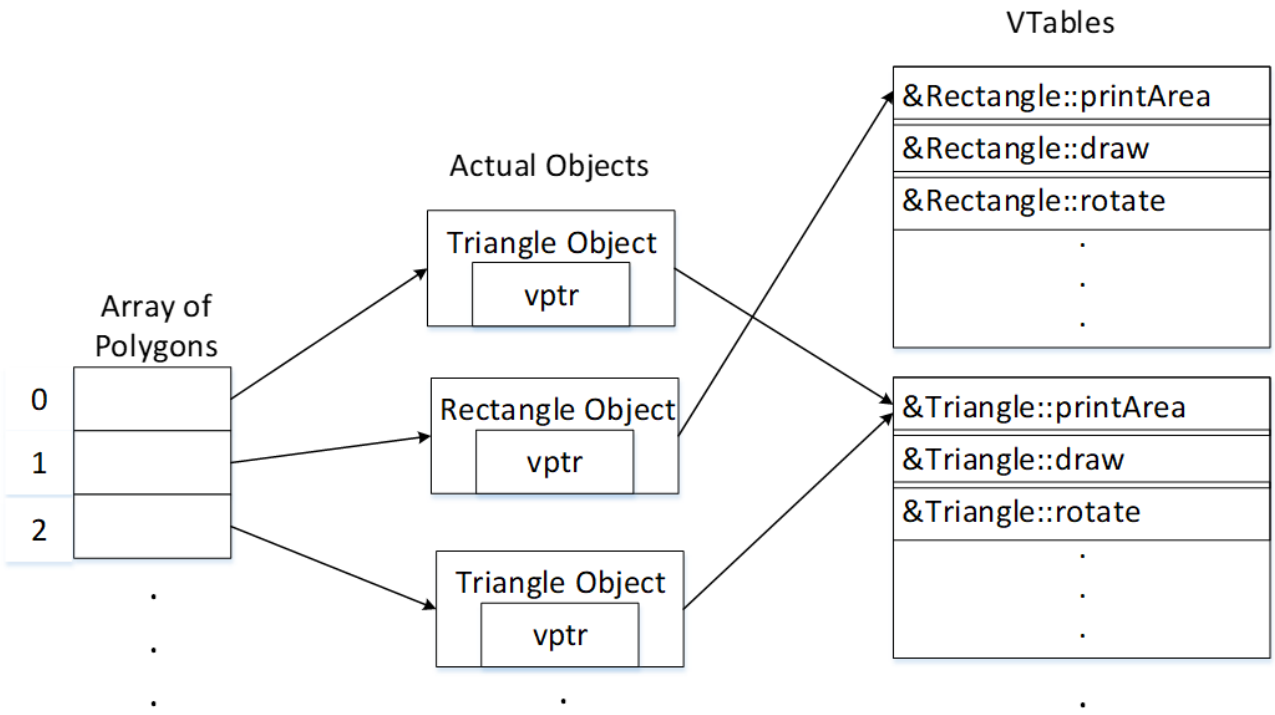
vtable: A table of function pointers.

It is maintained per class.

vptr: A pointer to vtable.

It is maintained per object.

- Compiler adds additional code at two places to maintain and use *vptr*.
    - **Code in every constructor. This code sets vptr of the object being created. This code sets *vptr* to point to *vtable* of the class.**
    - **Code with polymorphic function call.**
- Wherever a polymorphic call is made, compiler inserts code to first look for *vptr*.
- Once *vptr* is fetched, *vtable* of derived class can be accessed. Using *vtable*, address of derived class.



44 - Object Oriented Programming - Implementing Composition

ALI ABBAS

Overview

In this topic, you will learn the implementation of composition

Learning Goals

- Implementation of composition

OOP – Implementing Composition

```
class Time{

public:

    Time();

    Time(int, int);

    // some setters, getters, and utility functions

    void printTime();
```

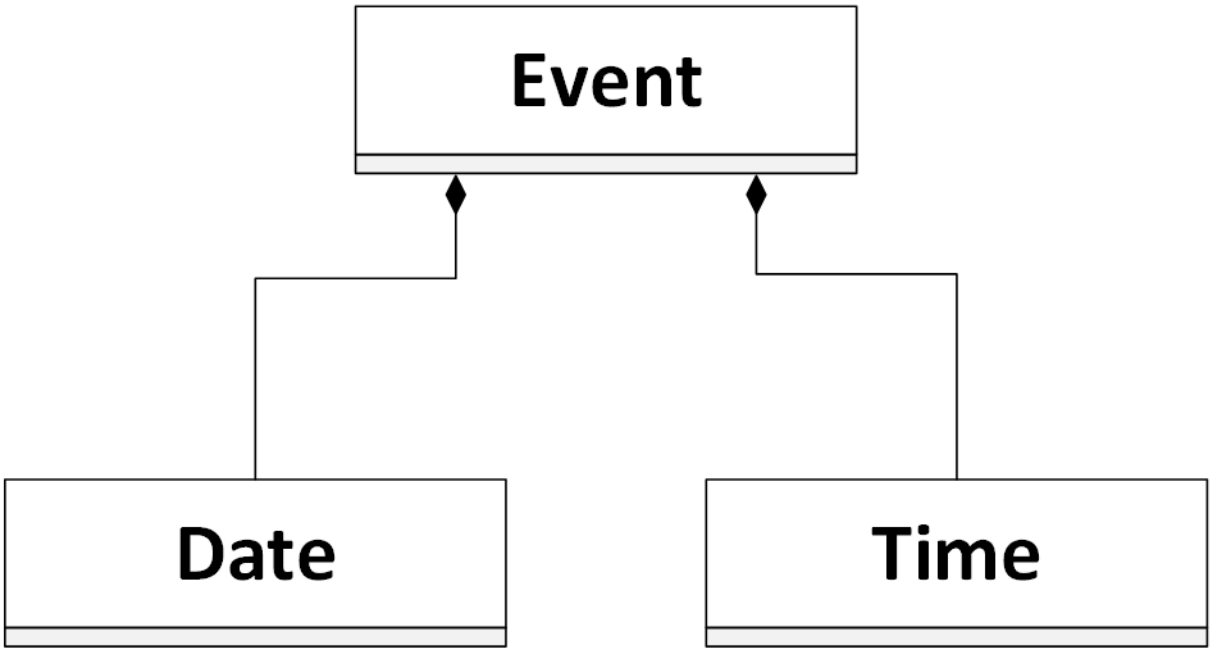
```
private:
    int hr;
    int min; };
class Date{
public:
    Date();
    Date(int, int, int);
    // some setters, getters, and utility functions
    void printDate();
private:
    int month;
    int day;
    int year;
};
class Event {
public:
Event(int hours = 0, int minutes = 0,
            int m = 1, int d = 1, int y = 1900,
            string name = “Start of 20th Century");
// setters, getters, and other utility functions
    void printEventData();
private:
    string eventName;
```

ALI ABBAS

```
    Time eventTime;
    Date eventDay;
};
int main(){
    //instantiate an object and set data for Eid prayer
    Event eidPrayer(6, 30, 12, 8, 2019, "Eid Prayer");
    eidPrayer.printEventData();
                //print out the data for object
//instantiate the second object
    Event independenceDay(8, 0, 14, 8, 2019, "National Anthem");
    independenceDay.printEventData();
                //print out the data for the second object
     return 0;
}
Event::Event(int hours, int minutes,
                 int m, int d, int y, string name)
                : eventTime(hours, minutes),
                eventDay(m, d, y)
{
    eventName = name;
}
void Event::printEventData()
{
```

ALI ABBAS

```
    cout << eventName << " occurs ";
    eventDay.printDate();
    cout << " at ";
    eventTime.printTime();
    cout << endl;
}
```



45 - Object Oriented Programming - Inheritance vs Composition

Overview

In this topic, you will learn the inheritance vs composition and advantage of inheritance over composition.

Learning Goals

- Learn about inheritance vs composition

OOP – Inheritance vs Composition

Inheritance

```
Derived_class_name::
Derived_class_constructor_name(types and parameters):
base_class_construcor_name(parameters)
{
   set parameters which are not set by the base
   class constructor;
}
```

Composition

```
Aggregate_class_name::
Aggregate_class_construcor_name(types and params):
component_class_instance(parameters)
{
   set parameters which are not set by the component
   class constructor;
}
```

- The main advantage inheritance has over composition is that it allows objects of the derived class to be dynamically bound in the same hierarchy as its base classes.
- It is also necessary where virtual methods must be overridden by the derived class
  - A composite aggregate cannot do this.

ALI ABBAS

46 - Implementing Uni-directional Associations

Overview

In this topic, you will learn implementing uni-directional association with code example.

Learning Goals

- Learn about implementation of uni-directional association

OOP – Implementing Uni-directional Associations

Example – Lecturer-Course Relationship

-
  - A lecturer may be assigned a course to teach. That lecturer may be removed from that course at anytime.

Unidirectional: lecturer is added to the course and not vice-versa

```
class course
{
private:
        lecturer * L      //        L is a pointer to an
                          //        object of type lecturer
public:
        course();
        void addlecturer(Lecturer *);
        void removelecturer();
};
```

ALI ABBAS

Lecturer and course are created separately and then linked together.

```
void course::addlecturer(Lecturer *teacher)
{
        L = teacher;
}
void course::removelecturer()
{
        L = NULL;
}
```

47 - Implementing Bi-directional Associations

Overview

In this topic, you will study about example of bi-directional associations.

Learning Goals

- Learn about the bi-directional associations

OOP – Implementing Bi-directional Associations

Example:

- A lecturer can be added to the course and vice versa

In a bi-directional associations both classes must include a reference(via a pointer) to the linked object.

Problem: User has to update both ends of the relationship.

Can be achieved by using the operator ***this.***

```
void course::addlecturer(lecturer *teacher)
{
    L = teacher;
    L -> addcourse(this);
}
void lecturer::addcourse(course *subject)
{
    C = subject;
}
void main()
```

ALI ABBAS

```
{
        course * SE101 = new course(…);
        lecturer *Fakhar = new lecturer(…);
        SE101->addlecturer(Fakhar);
        // the association is updated for both objects.
}
```

N:M associations are implemented using arrays of

pointers from one class to another.

What would happen if it could be added from either side?

```
void main()
{
        course * SE101 = new course(…);
        lecturer *Fakhar = new lecturer(…);

        Fakhar->addcourse(SE101);
}
void course::addlecturer(lecturer *teacher)
{
        L = teacher;
        L -> addcourse(this);
}
void lecturer::addcourse(course *subject)
{
```

ALI ABBAS

```
        C = subject;
        C -> addlecturer(this);
}
```

Control the infinite recursion

```
void course::addlecturer(lecturer *teacher)
{
        if (L == NULL) {
                L = teacher;
                L -> addcourse(this);
        }
}
void lecturer::addcourse(course *subject)
{
        if (C == NULL) {
                C = subject;
                C -> addlecturer(this);
        }
}
void main()
{
        course * SE101 = new course(…);
        lecturer *Fakhar = new lecturer(…);
        SE613->addlecturer(Fakhar);
```

ALI ABBAS

```
        // or
        // Fakhar->addcourse(SE101);
        ...
}
```

48 - SOLID Design Principles – Introduction

Overview

In this topic, you will learn an example based on Single Responsibility Principle (SRP) as part of SOLID Design Principles

Learning Goals

- Learn about SOLID design principles (Single Responsibility Principle (SRP))

SOLID Design Principles:

Single Responsibility Principle (SRP) – Example

Multiple Responsibilities

```
class Post
{
   void CreatePost(Database db, string postMessage)
   {
      try
      {
         db.Add(postMessage);
      }
      catch (Exception ex)
```

ALI ABBAS

```
        {
            db.LogError("An error occured: ", ex.ToString());
            File.WriteAllText("\LocalErrors.txt", ex.ToString());
        }
    }
}
```

Class for handling error logging

```
class ErrorLogger
{
    void log(string error)
    {
     db.LogError("An error occured: ", error);
     File.WriteAllText("\LocalErrors.txt", error);
    }
}
```

ALI ABBAS

Single Responsibility!

```
class Post
{
    private ErrorLogger errorLogger = new ErrorLogger();
    void CreatePost(Database db, string postMessage)
    {
        try
        {
```

```
            db.Add(postMessage);
        }
        catch (Exception ex)
        {
            errorLogger.log(ex.ToString());
        }
    }
}
```

49 - SOLID Design Principles - Single Responsibility Principle (SRP)

ALI ABBAS

Overview

In this topic, you will study first SOLID principle, which is single responsibility principle (SRP).

Learning Goals

- Learn about single responsibility principle

SOLID Design Principles

Single Responsibility Principle (SRP)

A class should have only one reason to change.

- **Cohesion**
  - Defined as the functional relatedness of the elements of a module
  - Related to the forces that cause a module, or a class, to change

Responsibility - a reason for change

- **What a class does**
  - The more a class does, the more likely it will change
  - The more a class changes, the more likely we will introduce bugs

SRP – Coupling and Cohesion

- If a class has more than one responsibility
  - Low cohesion
  - High coupling - the responsibilities become coupled
    - Changes to one responsibility may impair or inhibit the class's ability to meet the others
    - Leads to fragile designs that break in unexpected ways when changed

SRP

- One of the simplest of the principles but one of the most difficult to get right.
- Finding and separating conjoined responsibilities is much of what software design is really about
- The rest of the principles come back to this issue in one way or another

50 - Single Responsibility Principle (SRP) – Example

ALI ABBAS

Overview

In this topic, you will learn an example based on Single Responsibility Principle (SRP) as part of SOLID Design Principles

Learning Goals

- Learn about SOLID design principles (Single Responsibility Principle (SRP))

SOLID Design Principles:

Single Responsibility Principle (SRP) – Example

Multiple Responsibilities

```
class Post

{

   void CreatePost(Database db, string postMessage)

   {
```

```
        try
        {
            db.Add(postMessage);
        }
        catch (Exception ex)
        {
            db.LogError("An error occured: ", ex.ToString());
            File.WriteAllText("\LocalErrors.txt", ex.ToString());
        }
    }
}
```

ALI ABBAS

Class for handling error logging

```
class ErrorLogger
{
    void log(string error)
    {
      db.LogError("An error occured: ", error);
      File.WriteAllText("\LocalErrors.txt", error);
    }
}
```

Single Responsibility!

```
class Post
{
```

```
    private ErrorLogger errorLogger = new ErrorLogger();
    void CreatePost(Database db, string postMessage)
    {
        try
        {
            db.Add(postMessage);
        }
        catch (Exception ex)
        {
            errorLogger.log(ex.ToString());
        }
    }
}
```

ALI ABBAS

51 - SOLID Design Principles - Open-Closed Principle (OCP)

Overview

In this topic, you will study about open close principle. In open close principle, a software artifact should be open for extension but closed for modification. Modules that conform to the open-closed principle have two primary attributes, open for extension but close for modification.

Learning Goals

- Learn about open close principle

SOLID Design Principles

Open-Closed Principle (OCP)

The Open-Closed Principle (OCP):

- Coined by Bertrand Meyer:
- A software artifact should be open for extension but closed for modification.

The behavior of a software artifact ought to be extendible, without having to modify that artifact.

- Not simply a principle that guides in the design of classes and modules.
- Fundamental to good architectural design

If simple extensions to the requirements force massive changes to the software, then the architects of that software system have engaged in a spectacular failure.

- Modules that conform to the open-closed principle have two primary attributes.
- They are "Open For Extension".
    - We can make the module behave in new and different ways as the requirements of the application change, or to meet the needs of new applications.
- They are "Closed for Modification".
    - The source code of such a module is inviolate.
    - No one is allowed to make source code changes to it.
- Open-Close - Are these two attributes are at odds with each other?
    -
        - The normal way to extend the behavior of a module is to make changes to that module.
        - A module that cannot be changed is normally thought to have a fixed behavior.
        - How can these two opposing attributes be resolved?

Extension through inheritance, composition, and association

ALI ABBAS

52 - Open-Closed Principle (OCP) – Example

Overview

In this topic, you will get code example of open close principle.

Learning Goals

- Practical implementation of open close principle

SOLID Design Principles

Open-Closed Principle (OCP) - Example

```
class Post
{
   void CreatePost(Database db, string postMessage)
   {
      if (postMessage.StartsWith("#"))
      {
         db.AddAsTag(postMessage);
      }
      else
      {
         db.Add(postMessage);
      }
   }
}
```

Case for #

Otherwise

What Happens if we had to add a case for @

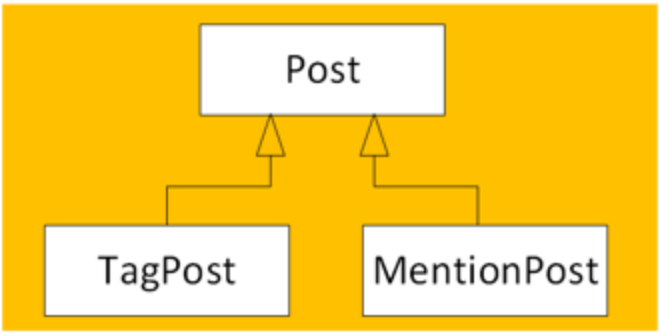
Violates OCP

```
class Post
{
   void CreatePost(Database db,
                   string postMessage)
   {
      db.Add(postMessage);
   }
}
```

```
class TagPost
{
   override void CreatePost(Database db,
                            string postMessage)
   {
      db.AddAsTag(postMessage);
   }
}
```

Now OCP Compliant

Can easily add a new post type



53 - Liskov's Substitution Principle (LSP)

Overview

In this topic, you will know about liskov substitution principle (LSP). In LSP, functions that use references to base classes must be able to use objects of derived classes without knowing it.

Learning Goals

- Learn about liskov substitution principle

SOLID Design Principles

Liskov's Substitution Principle (LSP)

Liskov Substitution Principle (LSP)

- Subtypes must be substitutable for their base types.
- Deals with design of inheritance hierarchies

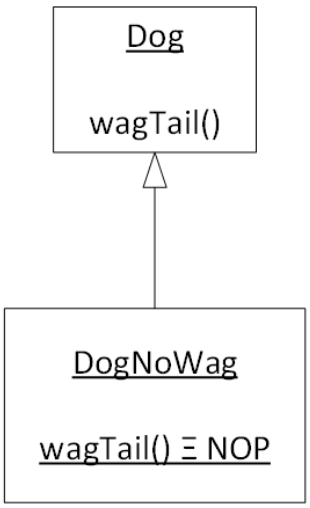
Functions that use references to base classes must be able to use objects of derived classes without knowing it.

ALI ABBAS

All dogs know how to wag their tails.

DogNoWag is a special type of dog.

DogNoWag does not know how to wag its tail.



Ignoring the functionality defined in the base class is violation of LSP

ALI ABBAS

54 - Liskov's Substitution Principle (LSP) – Example

Overview

In this topic, you will study the code example of liskov substitution principle.

Learning Goals

- Practical implementation of liskov substitution principle

SOLID Design Principles

Liskov's Substitution Principle (LSP) - Example

```
class Rectangle

{

public:
```

```
virtual void SetWidth(double w) {itsWidth=w;}

virtual void SetHeight(double h) {itsHeight=w;}

double GetHeight() const {return itsHeight;}

double GetWidth() const {return itsWidth;}

private:

double itsWidth;

double itsHeight;

};
```

ALI ABBAS

```
class Square : public Rectangle
{
public:
    virtual void SetWidth(double w);
    virtual void SetHeight(double h);
};

void Square::SetWidth(double w)
{
    Rectangle::SetWidth(w);
    Rectangle::SetHeight(w);
}
```

SetHeight will be the same as SetWidth

By setting the width of a Square object, its height will change correspondingly and vice versa.

The Square object will remain a mathematically proper square

Good? – Not really!

```
void g(Rectangle& r)
{
    r.SetWidth(5);
    r.SetHeight(4);
    assert(r.GetWidth() * r.GetHeight()) == 20);
}
```

Works fine for a rectangle
but
assertion error if passed a square!

A violation of LSP!

55 - The True Meanings of IsA Relationship

Overview

In this topic, you will study the meaning of IsA relationship while considering liskov substitution principle.

Learning Goals

- Develop understanding of the IsA relationship meaning

SOLID Design Principles

Liskov's Substitution Principle and Inheritance

The True Meanings of IsA Relationship

Validity is not Intrinsic

ALI ABBAS

- A model, viewed in isolation, cannot be meaningfully validated.
- The validity of a model can only be expressed in terms of its clients.
- Square and Rectangle classes viewed in isolation, appeared to be self-consistent and valid.
- Viewed from a programmer standpoint who made reasonable assumptions about the base class, the model broke down.
- When considering whether a particular design is appropriate or not, one must not simply view the solution in isolation.
    - One must view it in terms of the reasonable assumptions that will be made by the users of that design.

What Went Wrong? (W^3)

- Isn't a Square a Rectangle?
- Doesn't the ISA relationship hold?
- No!
- A square might be a rectangle, but a Square object is definitely not a Rectangle object.
- Why?
    - Because the behavior of a Square object is not consistent with the behavior of a Rectangle object.
    - Behaviorally, a Square is not a Rectangle!

And it is behavior that software is really all about!

- The LSP makes clear that in OOD the ISA relationship pertains to behavior.
    - Not intrinsic private behavior, but extrinsic public behavior;
    - Behavior that clients depend upon.

56 - SOLID Design Principles - Interface Segregation Principle (ISP)

Overview

- In this topic, you will know about interface segregation principle. Clients should not be forced to depend upon interfaces that they do not use. If you have an abstract class or an interface, then the implementers should not be forced to implement parts that they do not care about.

Learning Goals

- Learn about interface segregation principle

ALI ABBAS

57 - Interface Segregation Principle (ISP) – Example

Overview

In this topic, you will know about the example of interface segregation principle.

Learning Goals

- Practical implementation of interface segregation principle

58 - SOLID Design Principles - Dependency Inversion Principle (DIP)

Overview

In this topic, you will know about the dependency inversion principle. This principle guides that high-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend upon details. Details should depend upon abstractions.

Learning Goals

- Learn about the dependency inversion principle

59 - Dependency Inversion Principle (DIP) – Example

Overview

In this topic, you will know about the code example of dependency inversion principle.

Learning Goals

- Practical implementation of dependency inversion principle

ALI ABBAS

60 - Good OO Design Principles - Law of Demeter (LoD)

Overview

In this topic, you will know about the explanation of law of Demeter and aim. It is a Style Rule for building systems. The Demeter Research Group proposes it in 1987. The Law of Demeter says that if I need to request a service of an objects sub-part, I should instead make the request of the object itself and let it propagate this request to all relevant sub-parts, thus the object is responsible for knowing its internal make-up instead of the method that uses it.

Learning Goals

- Learn about the law of Demeter

61 - Law of Demeter (LoD) – Example

Overview

In this topic, detailed example of law of Demeter will be discussed.

Learning Goals

- Develop understanding of law of Demeter with an example

62 - Action-Oriented Design in the Disguise of Object-Oriented Design

Overview

In this topic, you will get an understanding about action oriented design, in which you will be described about objects and classes. Good design practices, design Challenges with OO Paradigm and god class problem will also be discussed.

Learning Goals

- Learn about the action oriented design

ALI ABBAS

63 - Handling Poorly Distributed System Intelligence

Overview

In this topic, you will study about handling poorly distributed system intelligence and how to combine intelligence in one place.

Learning Goals

- Develop understanding of the poorly distributed system intelligence
- How to combine intelligence in one place

64 - Modelling Policy – Jacobson's Heuristic

Overview

In this topic, you will study about Jacobson's heuristic in which, you will learn that a policy information should not place inside the classes involved in decision making.

Learning Goals

- Develop understanding of the modeling policy

65 - Handling Roles - Player Role Pattern

Overview

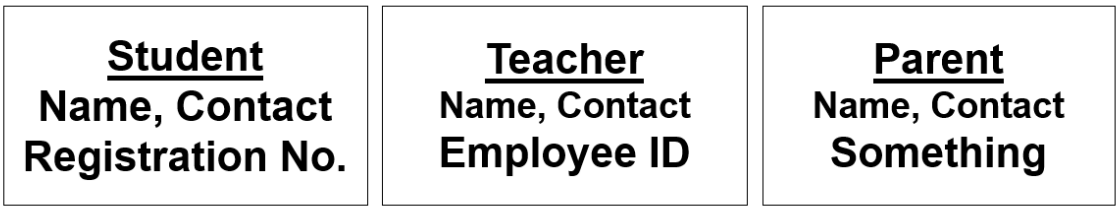
In this topic, you will study about a problem in which a person playing different roles, then a solution using player role pattern will be given.

Learning Goals

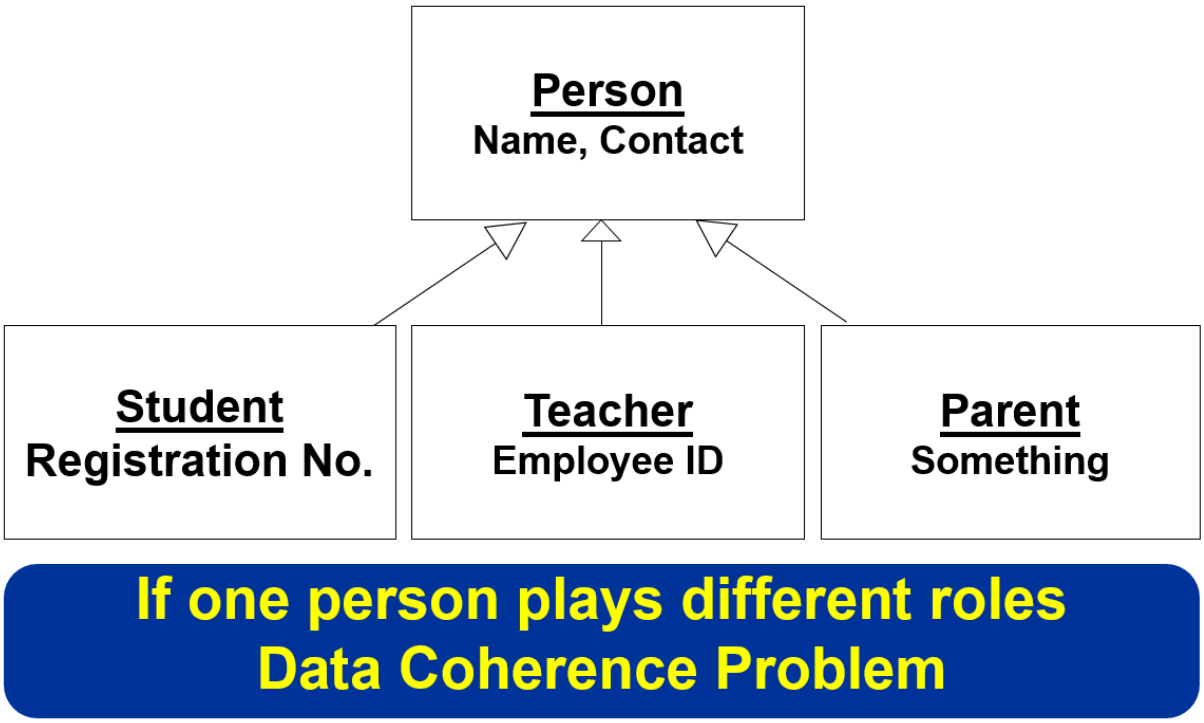
- Develop understanding about different roles of a person
- Solution of the problem using player role pattern

ALI ABBAS

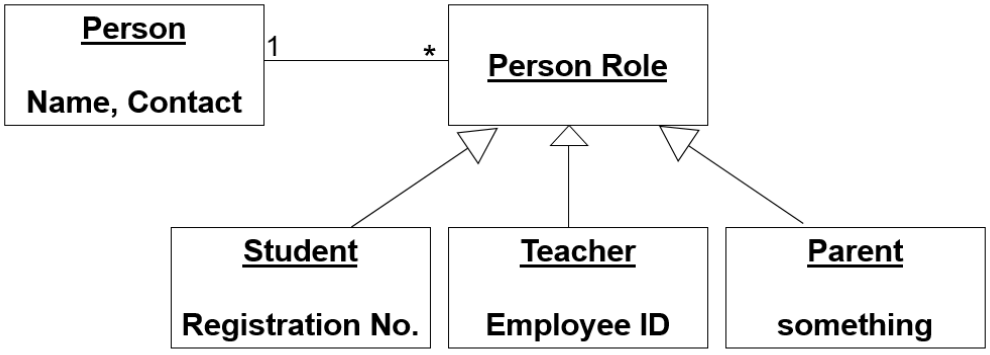
Handling roles - Player-Role Pattern



Roles



Role Object



66 - Handling Multiple Discriminators Using Player-Role Pattern

Overview

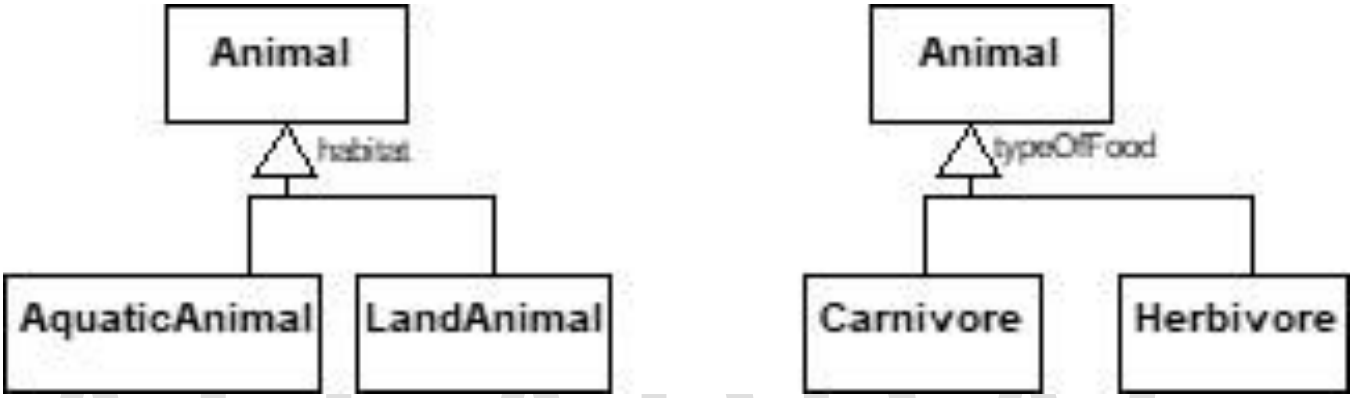
In this topic, you will get understanding of handling multiple discriminators using Player-Role Pattern, Creating higher-level generalization and Using Multiple Inheritance. A discriminator is a label that describes the criteria used in the specialization.

Learning Goals

- Learn about the using of player role pattern

Handling multiple discriminators using Player-Role Pattern

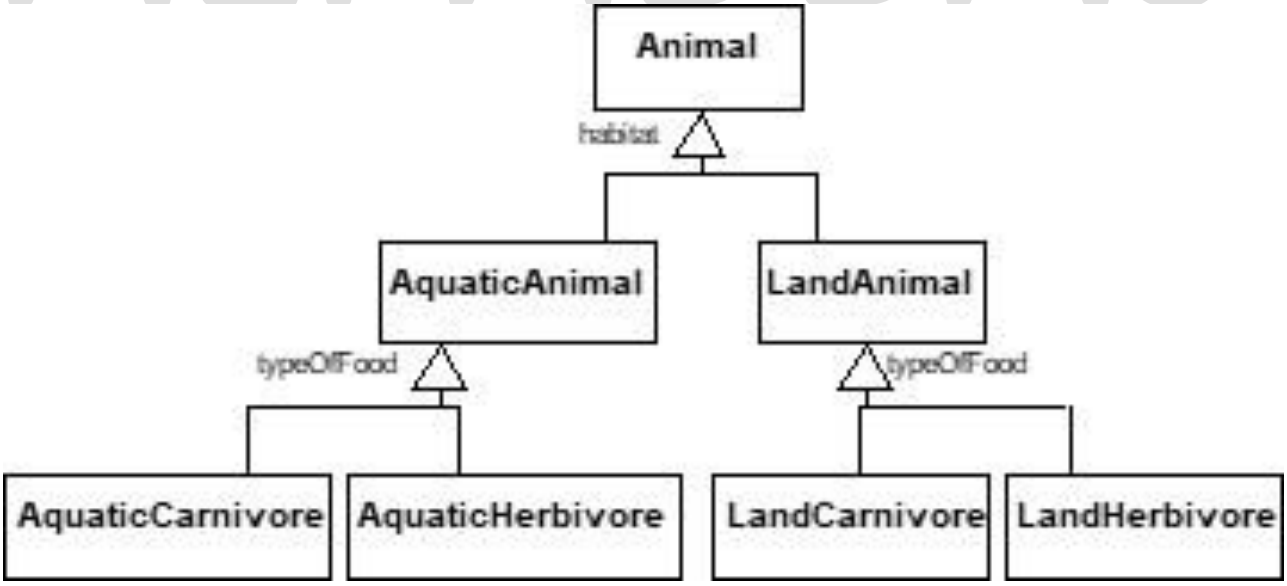
A discriminator is a label that describes the criteria used in the specialization. It can be thought of an attribute that will have a different value in each subclass.



Handling multiple discriminators by creating higher-level generalization

ALI ABBAS

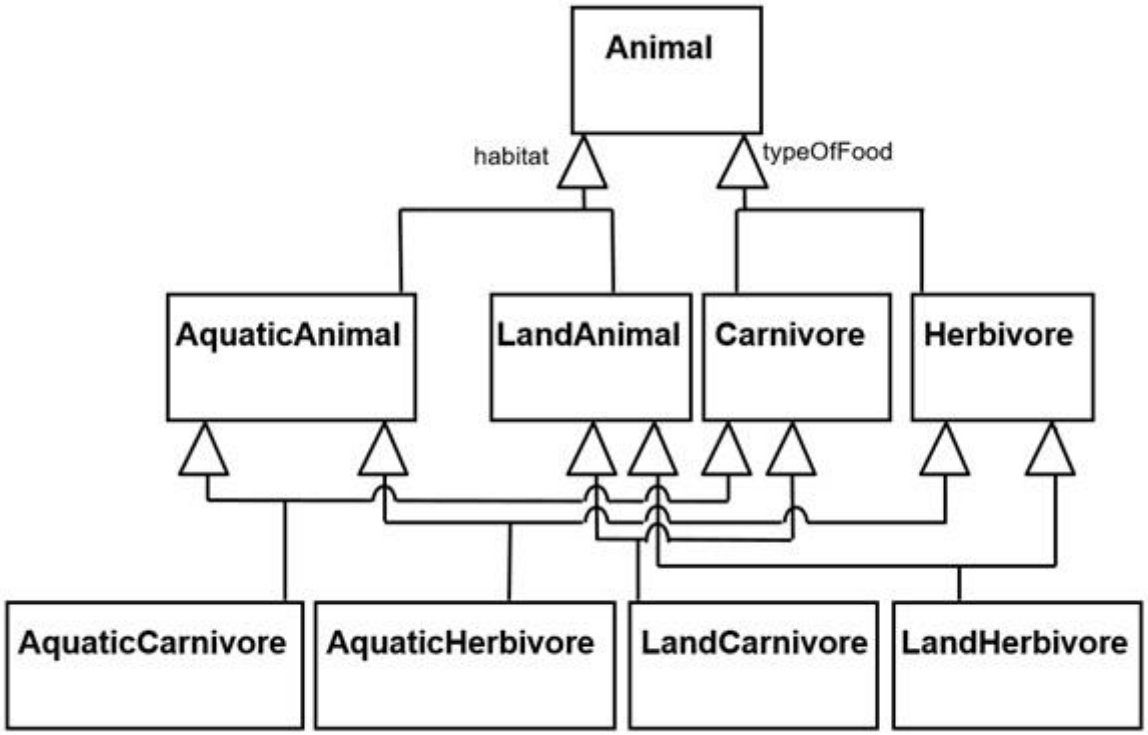
- Properties associated with both discriminators have to be present
- All the features associated with the second generalization set would have to be duplicated.
- No. of classes can grow very large try adding omnivores and/or amphibians!



Handling multiple discriminators by Using Multiple Inheritance

- Avoids duplication of features

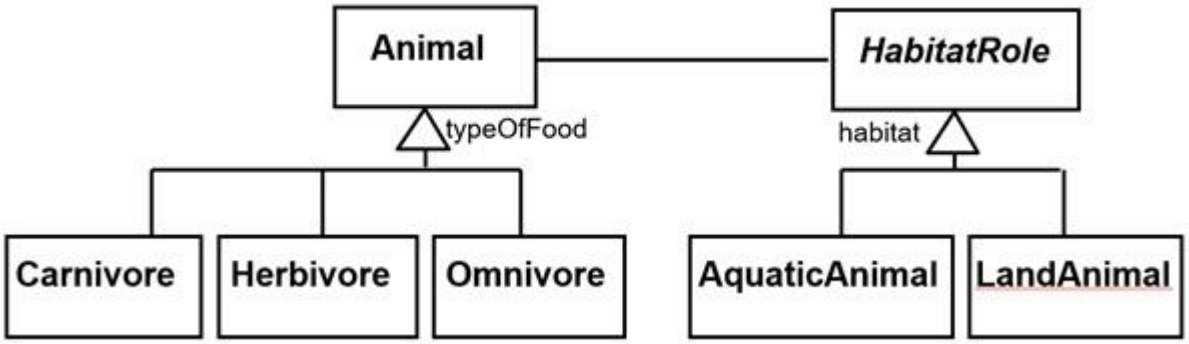
- Adds too much complexity - Proliferation of classes not supported by many languages like Java and C#



Handling multiple discriminators by Using Player-Role Pattern

ALI ABBAS

- Can very easily add Amphibian



67 - Software Design and Architecture - Abstraction-Occurrence Pattern

Overview

In this topic, you will get the understanding of abstraction-occurrence pattern with examples.

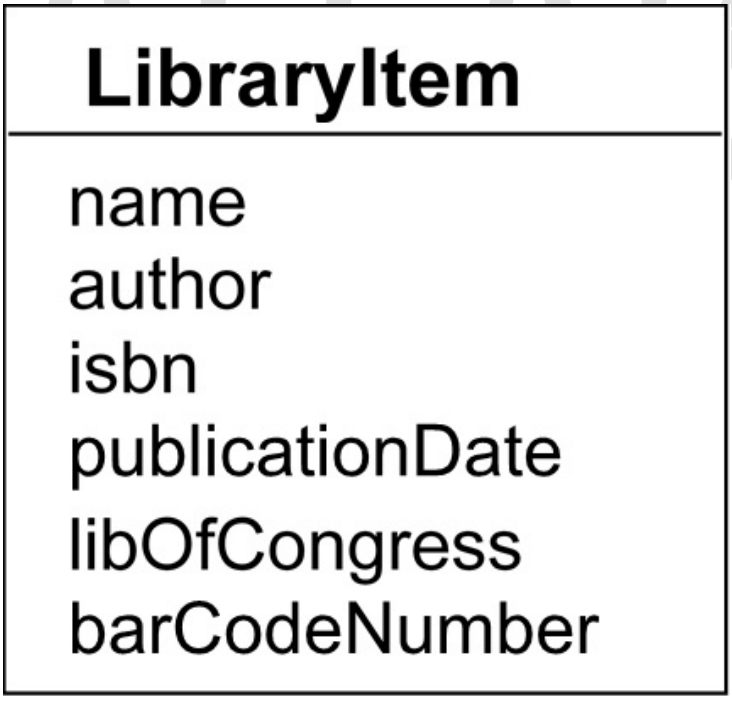
Learning Goals

- Learn about the abstraction-occurrence pattern

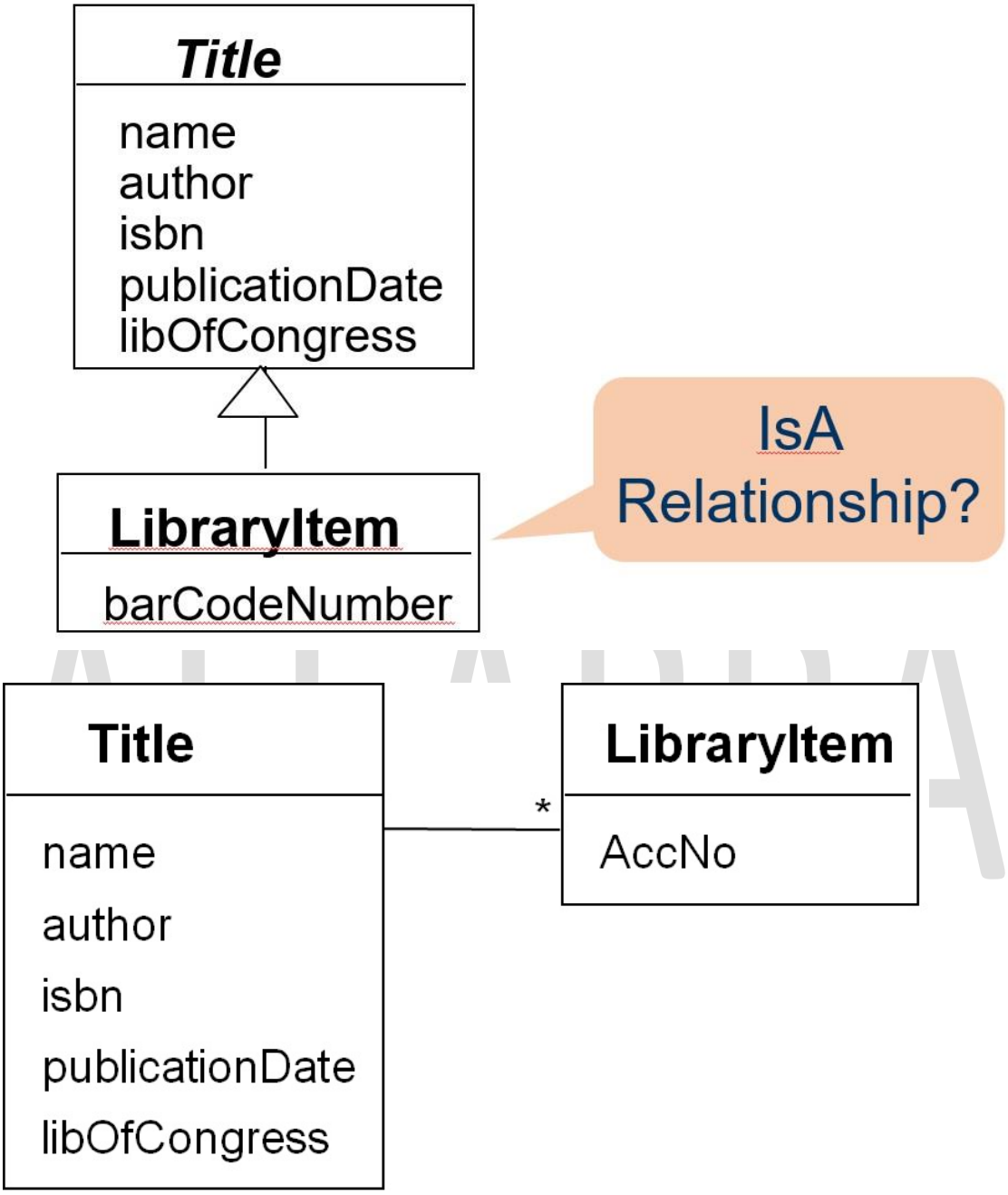
Abstraction-Occurrence Pattern

Problem

-
  - A library has multiple copies of the same book.
  - How to model?



ALI ABBAS

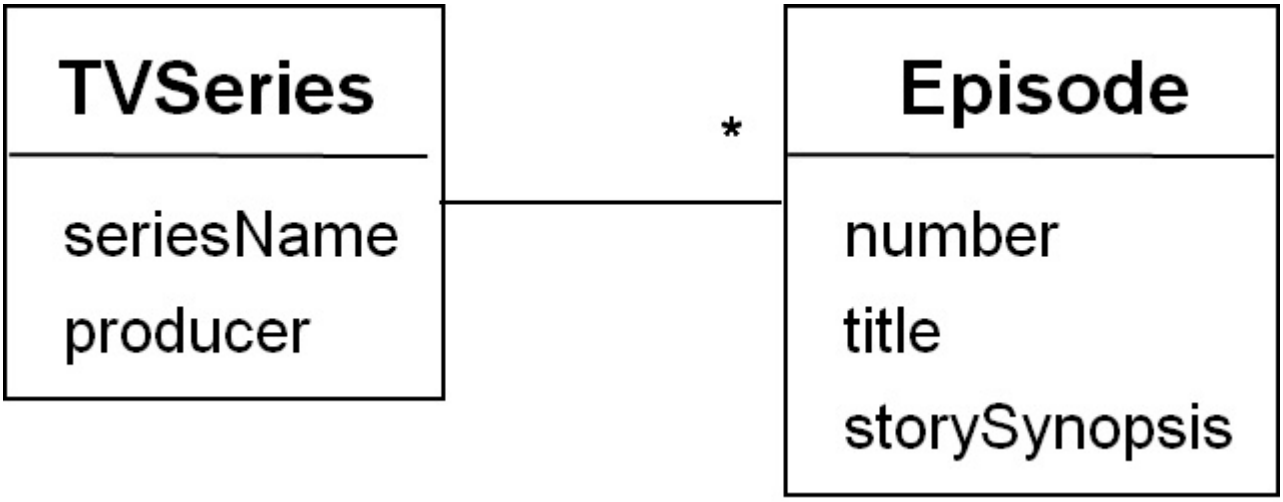


The Abstraction-Occurrence Pattern

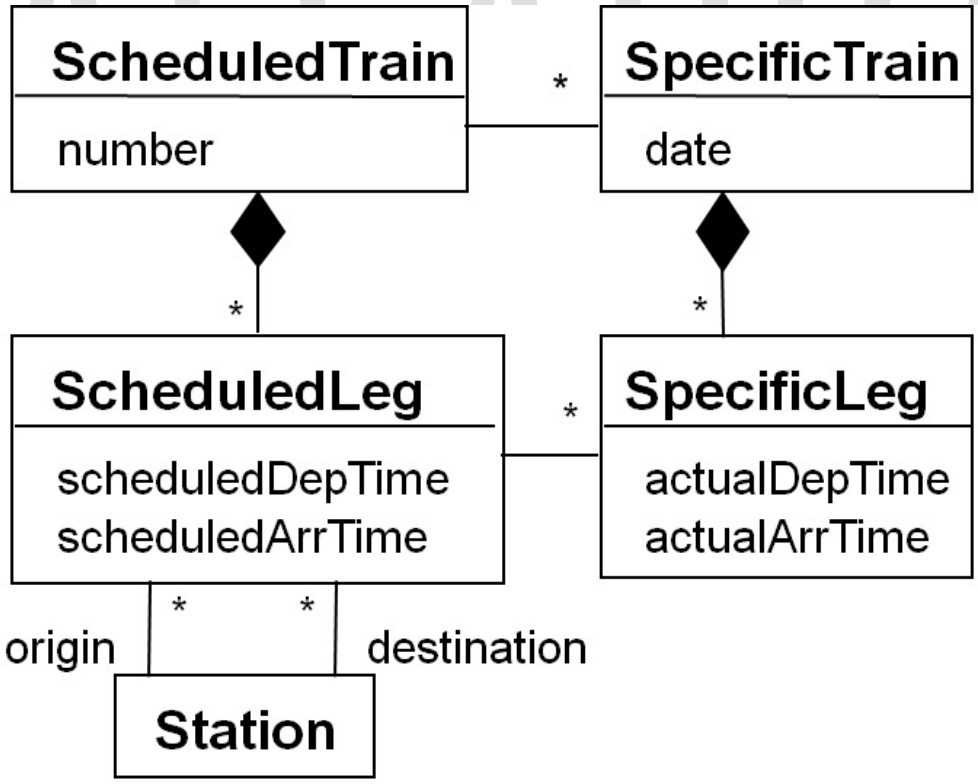
- ***Context***:
  - Often in a domain model you find a set of related objects (*occurrences)*.
  - The members of such a set share common information
    - but also differ from each other in important ways.
- ***Problem***:
  - What is the best way to represent such sets of occurrences in a class diagram?
- ***Forces***:

- You want to represent the members of each set of occurrences without duplicating the common information

Abstraction-Occurrence – Examples



Square variant



ALI ABBAS

68 - Software Design and Architecture - Reflexive Associations

Overview

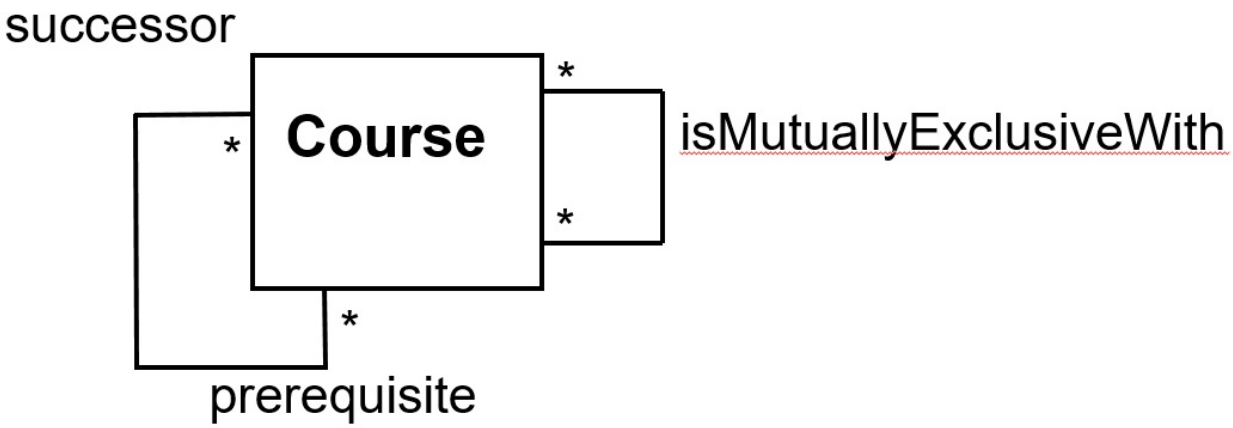
In this topic, you will get the understanding of reflexive association and its types. An association to connect a class to itself. Two types of reflexive association are Asymmetric Reflexive Association and Symmetric Reflexive Association.

Learning Goals

- Learn about the reflexive association
- Learn about the types of reflexive association

Reflexive Associations

- An association to connect a class to itself.
- Two main types: Symmetric and Asymmetric.



Asymmetric Reflexive Association

- The ends of the association are semantically different from each other, even though the associated class is the same.
- Examples:
  - parent-child, supervisor-subordinate, and predecessor-successor, course-prerequisite

```
// Example - asymmetric association

class Person {

Person *parents[2];
```

…

};

// Example - asymmetric association with **role name**

// on the other end

```
class Person {
  Person *parents[2];
  vector <Person *> child;
  …
}
```

Symmetric Reflexive Association

- There is no logical difference in the semantics of each association end
- Example:
  - mutually exclusive courses
    - students who have taken one course cannot take another in the set
  - If course A is mutually exclusive with B, then course B is mutually exclusive with A

ALI ABBAS

// Example symmetric association.

```
class Course {
    Course  *mutuallyExclusiveWith[3];
};
```

69 - Software Design and Architecture - Design Patterns Introduction

Overview

In this topic, you will get the understanding of design patterns.

Learning Goals

- Learn about the design patterns

Design Patterns – Introduction

- Good OOD is more than just knowing and applying concepts like abstraction, inheritance, and polymorphism
- A design guru thinks about how to create flexible designs that are maintainable and that can cope with change.

Design Patterns

"Each pattern describes a problem which occurs over and over again in our environment and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it in the same way twice"

- Christopher Alexander, A Pattern Language, 1977
- Context: City Planning and Building architectures

Software Design Patterns

- Search for recurring successful designs – emergent designs from practice
- Described in Gama, Helm, Johnson, Vlissides 1995 (i.e., "gang of 4 book" aka GoF)
- Represent solutions to problems that arise when developing software within a particular context.
- Describes recurring design structures
- Describes the context of usage

ALI ABBAS



What Makes it a Pattern?

- A Pattern must:
  - Solve a problem and be useful
  - Have a context and can describe where the solution can be used
  - Recur in relevant situations
  - Provide sufficient understanding to tailor the solution
  - Have a name and be referenced consistently

- Patterns capture the static and dynamic *structure* and *collaboration* among key participants in software designs
- Especially good for describing how and why to resolve *non-functional issues*
- Patterns facilitate reuse of successful software architectures and designs.

70 - Software Design and Architecture - Elements of Design Pattern

Overview

In this topic, you will get the understanding of the elements of design patterns. Design pattern have four essential elements: patter name, problem, solution and consequences.

ALI ABBAS

Learning Goals

- Learn about the elements of design patterns

Elements of design patterns

Design patterns have four essential elements:

-
  - Pattern name
  - Problem
  - Solution
  - Consequences

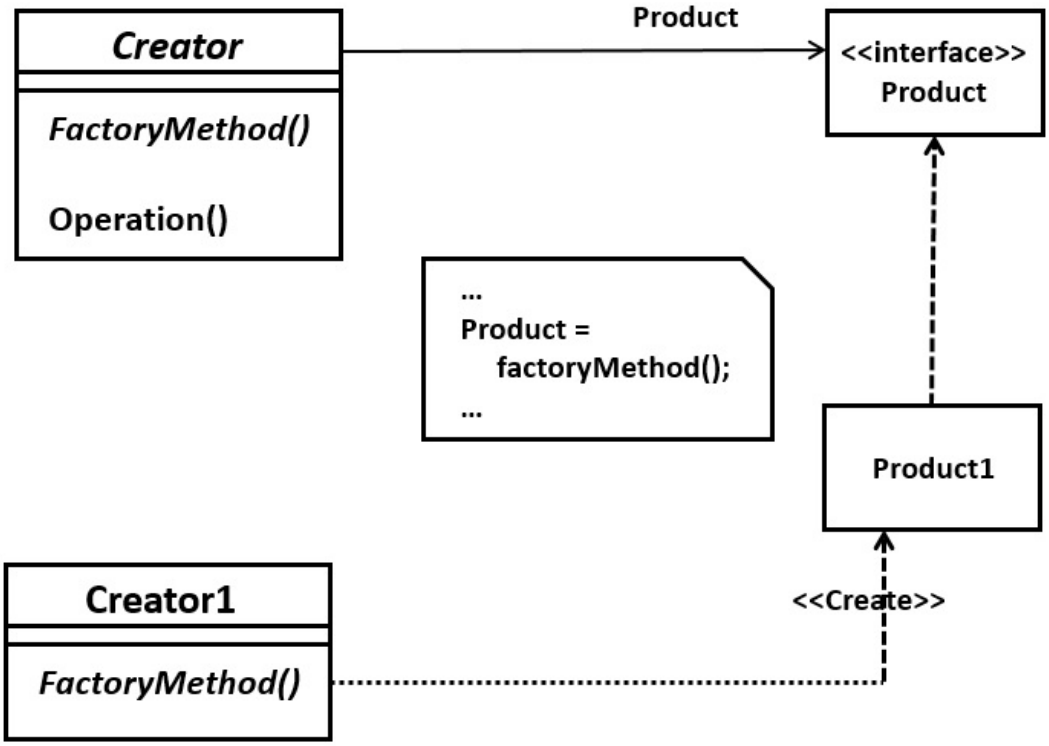
Pattern Name

A handle used to describe:

-
  - a design problem
  - its solutions
  - its consequences
- Increases design vocabulary
- Makes it possible to design at a higher level of abstraction
- Enhances communication

- *"The Hardest part of programming is coming up with good variable [function, and type] names."*

Factory Method



Problem

Describes when to apply the pattern

-
  - Explains the problem and its context
  - May describe specific design problems and/or object structures
  - May contain a list of preconditions that must be met before it makes sense to apply the pattern

Solution

- Describes the elements that make up the
  - design
  - relationships
  - responsibilities
  - collaborations

- Abstract description of design problems and how the pattern solves it
- Does not describe specific concrete implementation

Consequences

- Results and trade-offs of applying the pattern
- Critical for:
  - evaluating design alternatives
  - understanding costs
  - understanding benefits of applying the pattern
- Includes the impacts of a pattern on a system's:
  - flexibility
  - extensibility
  - portability

Design Pattern Descriptions

ALIABBAS

- Name and Classification: Essence of pattern
- Intent: What it does, its rationale, its context
- AKA: Other well-known names
- Motivation: Scenario illustrates a design problem
- Applicability: Situations where pattern can be applied
- Structure: Class and interaction diagrams
- Participants: Objects/classes and their responsibilities
- Collaborations: How participants collaborate
- Consequences: Trade-offs and results
- Implementation: Pitfalls, hints, techniques, etc.
- Sample Code
- Known Uses: Examples of pattern in real systems
- Related Patterns: Closely related patterns

71 - Categories of Design Patterns

Overview

In this topic, you will get the introduction of design pattern categories. Introduction to types of patterns, class pattern and object pattern also discussed.

Learning Goals

- Learn about the design pattern categories
- Learn about the types of patterns

Categories of Design Patterns

Categories of Patterns

- **Creational patterns (5)**
  - Deal with initializing and configuring classes and objects
- **Structural patterns (8)**
  - Deal with decoupling interface and implementation of classes and objects
  - Composition of classes or objects
- **Behavioural patterns (11)**
  - Deal with dynamic interactions among societies of classes and objects
  - How they distribute responsibility

Pattern Types

- **Class Patterns (4)**
  - Deal with relationships between classes and their subclasses
  - Established through inheritance, so they are static-fixed at compile-time
- **Object Patterns (20)**
  - Deal with object relationships
  - Established through association
  - Can be changed at run-time and are more dynamic

ALI ABBAS

		Purpose		
		Creational	Structural	Behavioural
Scope	Class	• Factory method	• Adapter (class)	• Interpreter • Template method
	Object	• Abstract factory • Builder • Prototype • Singleton	• Adapter (object) • Bridge • Composite • Decorator • Façade • Flyweight • Proxy	• Chain of responsibility • Command • Iterator • Mediator • Memento • Observer • State • Strategy • Visitor

72 - Benefits and Drawbacks of Design Patterns

Overview

In this topic, you will be introduced about benefits and drawbacks of design patterns.

Learning Goals

- Learn about the benefits and drawbacks of design patterns

Benefits and drawbacks of design patterns

Benefits of Design Patterns

- Design patterns enable large-scale reuse of software architectures and also help document systems
- Patterns explicitly capture expert knowledge and design trade-offs and make it more widely available
- Pattern names form a common vocabulary
  - Patterns help improve developer communication
- Patterns help ease the transition to OO technology

ALI ABBAS

Drawbacks to Design Patterns

- Patterns do not lead to direct code reuse
- Patterns are deceptively simple
- Patterns are validated by experience and discussion rather than by automated testing
- Integrating patterns into a software development process is a human-intensive activity.

Design Patterns are NOT

- Designs that can be encoded in classes and reused as is (i.e., linked lists, hash tables)
- Complex domain-specific designs (for an entire application or subsystem)
- They are:
  - "Descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context."

73 - Singleton Design Pattern

Overview

In this topic, you will understand about the singleton design pattern that fall in the creational patterns category. Singleton pattern uses, structure and implementation will be discussed.

Learning Goals

- Learn about the singleton pattern

Singleton Design Pattern

The Singleton Pattern

- ***Context***:
  - It is very common to find classes for which only one instance should exist (*singleton)*
- ***Problem***:
  - How do you ensure that it is never possible to create more than one instance of a singleton class?
- ***Forces***:
  - The use of a public constructor cannot guarantee that no more than one instance will be created.
  - The singleton instance must also be accessible to all classes that require it

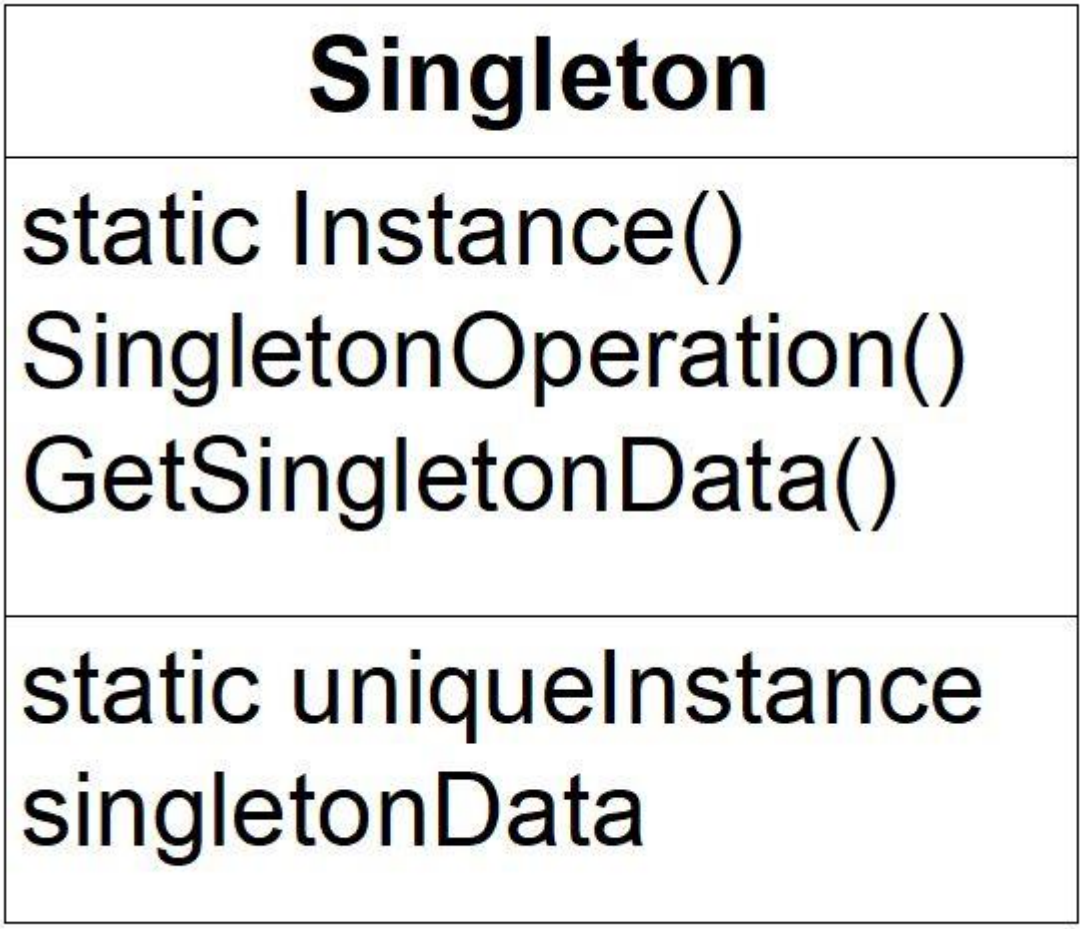
ALI ABBAS

Singleton – Uses

Use the Singleton when:

- There must be exactly one (or fixed number) instance of the class and it must be accessible to clients from a well-known access point

Singleton – Structure


Singleton
static Instance()
SingletonOperation()
GetSingletonData()
static uniqueInstance
singletonData

Implementation

```
Class Singleton {
public:
  static singleton* Instance();
protected:
  Singleton();
private:
  static singleton* _instance;
};
```

```
Singleton* Singleton::
                    _instance= NULL;

Singleton* Singleton::Instance() {
    if (_instance == NULL) {
      _instance = new Singleton;
    }
    return _instance;
}
```

74 - Strategy Design Pattern

ALI ABBAS

Overview

In this topic, you will understand about strategy design pattern, its definition and implementation. It defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Learning Goals

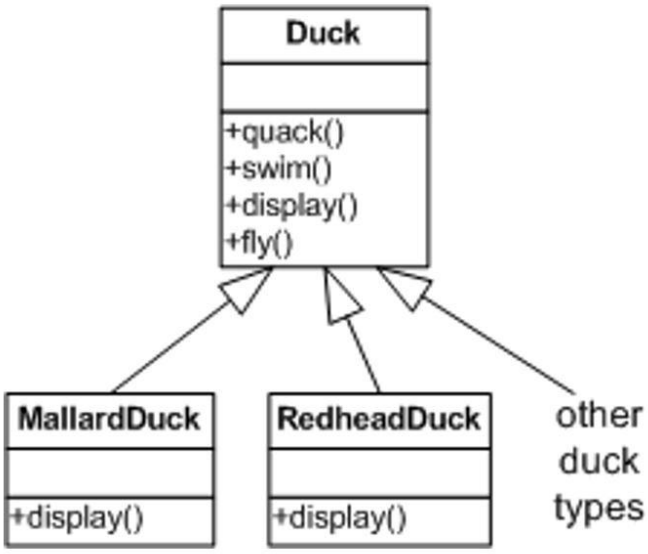
- Learn about the strategy design pattern

Strategy Design Pattern

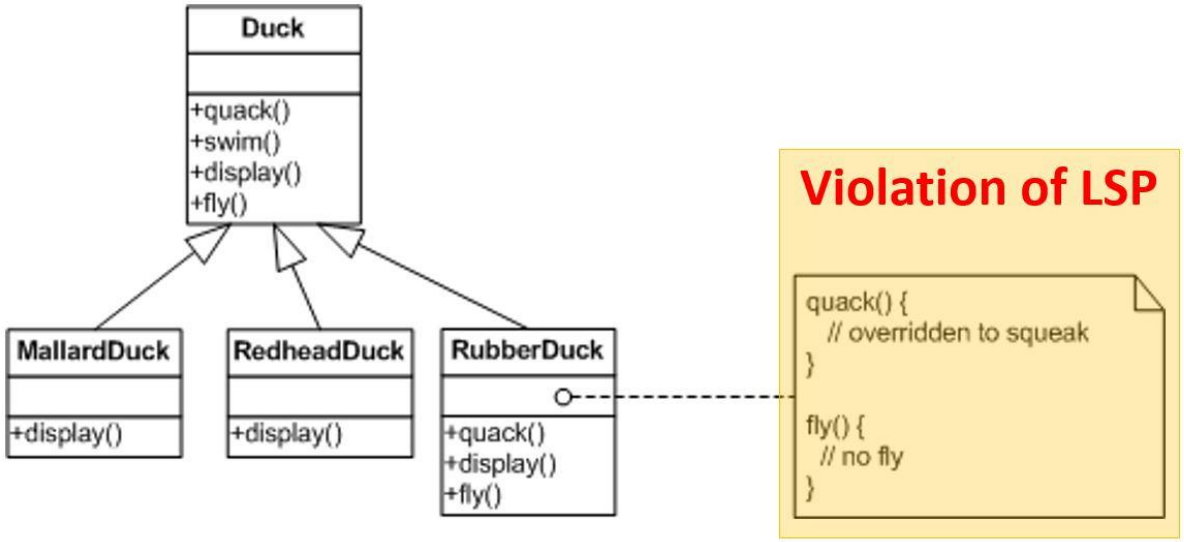
SimDuck

Add fly()

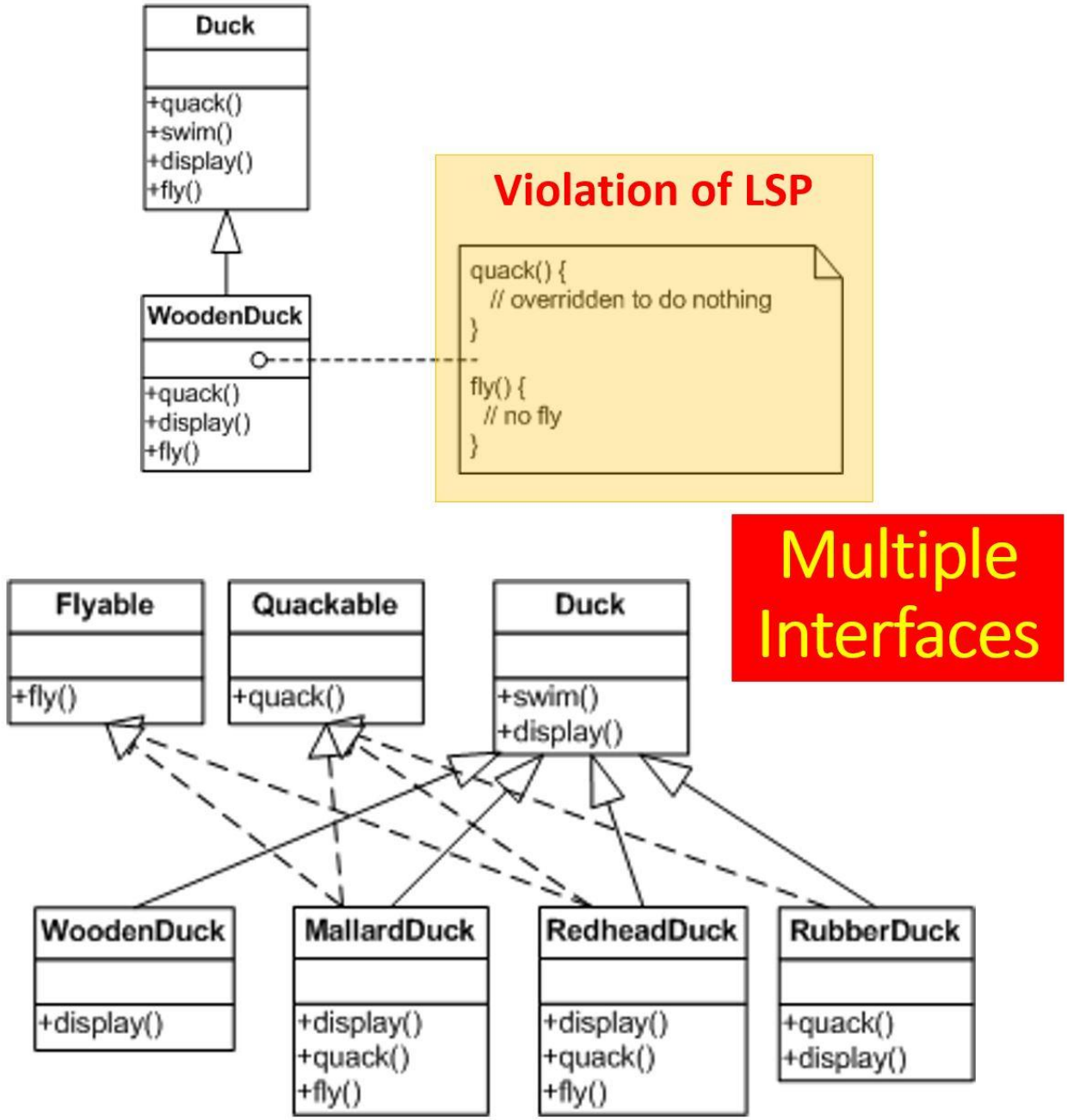
- **What about a rubber duck?**



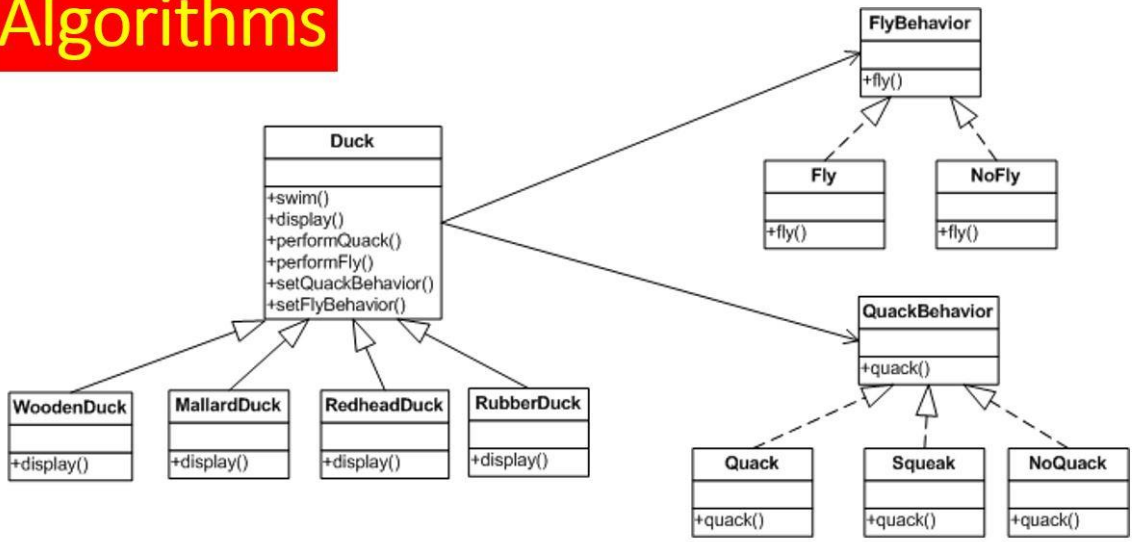
Rubber Duck



Another duck

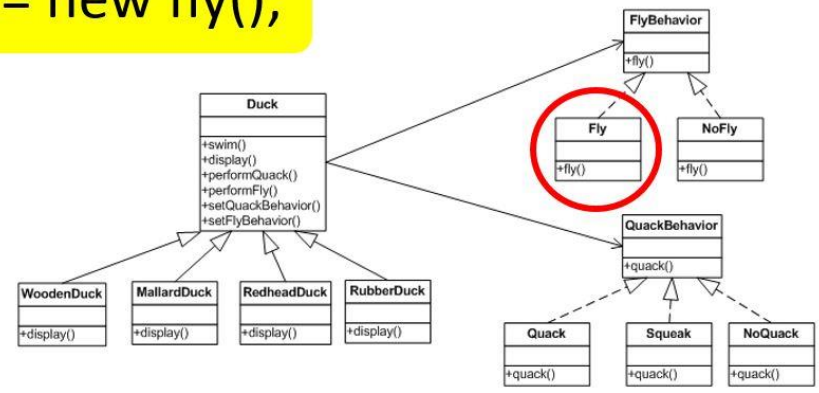


Associate Algorithms



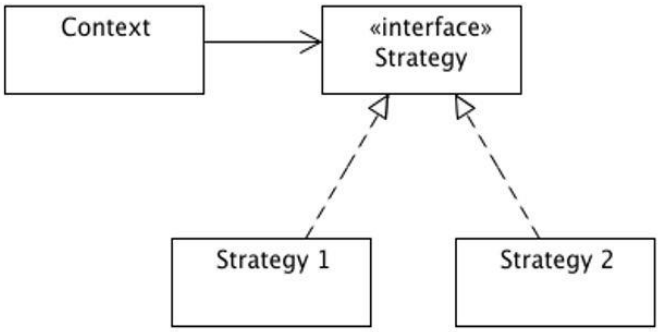

```
public abstract class Duck {
    FlyBehavior   flyBehavior;
    QuackBehavior quackBehavior;
    public Duck() { }
    public abstract void display();
    public void performFly() {
        flyBehavior.fly();
    }
    public void performQuack() {
        quackBehavior.quack();
    }
    // other
```

```
public class MallardDuck extends Duck {
 public MallardDuck() {
     quackBehavior = new Quack();
     flyBehavior = new fly();
 }
 // other
}
```



Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.



75 - Design Patterns - Introduction - Object-Morphing and Strategy Pattern

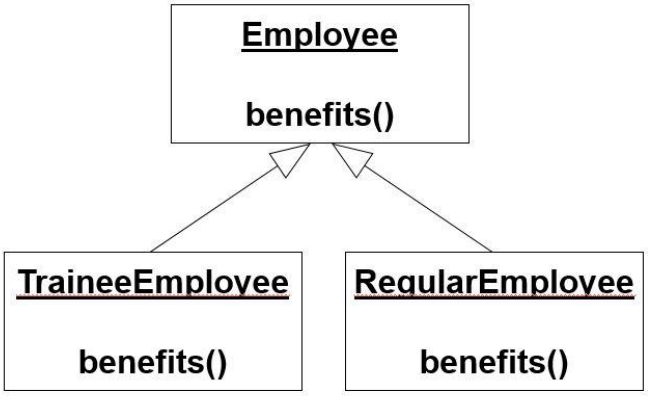
Overview

In this topic, you will understand about object morphing and its solution.

Learning Goals

- Learn about the object morphing

Object-Morphing and Strategy Pattern

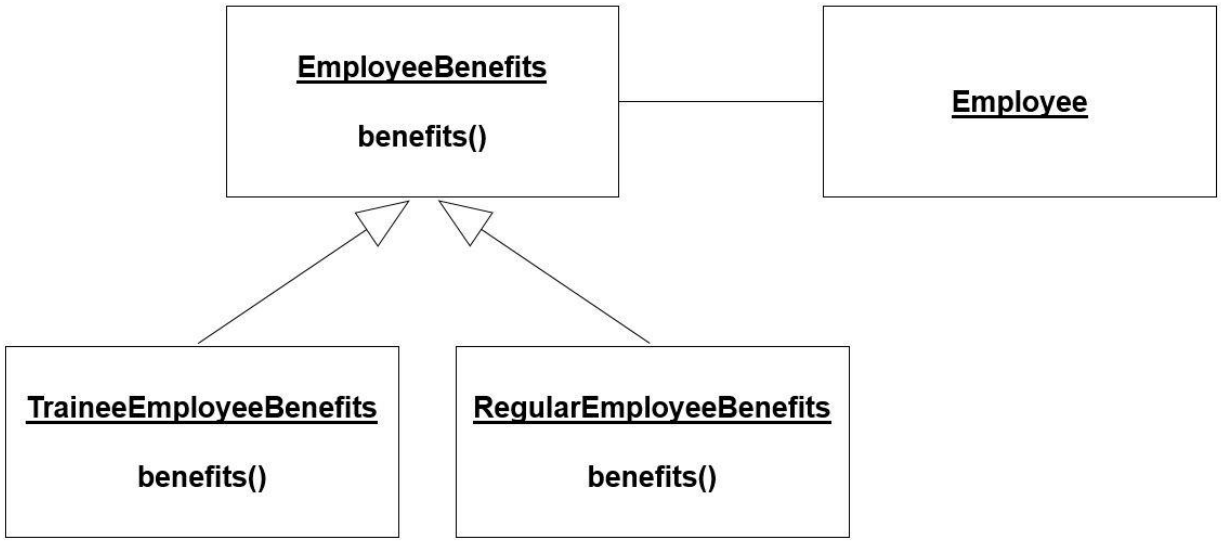


Everything seems satisfactory until we think about the life of the traineeEmployee object

Object Morphing

Avoiding having instances change class

- **Object Morphing**
  - When an instance changes its class
  - An instance should never need to change class
- **You have to destroy the old one and create a new one.**
  - What happens to associated objects?
    - You have to make sure that all links that connected to the old object now connect to the new one.



Object Morphing and Strategy Design Pattern

ALI ABBAS

76 - Facade Design Pattern

Overview

In this topic, you will be introduced about façade design pattern.

Learning Goals

- Learn about the façade design pattern

Façade Design Pattern

Problem

- There is a legacy application written in C
- There is a rich library of functionality that is very difficult to be re-written
- A new GUI-based interface needs to be added to the application
- The application needs to be “object-orientized” to make it communicate/compatible with the rest of a newly built system.

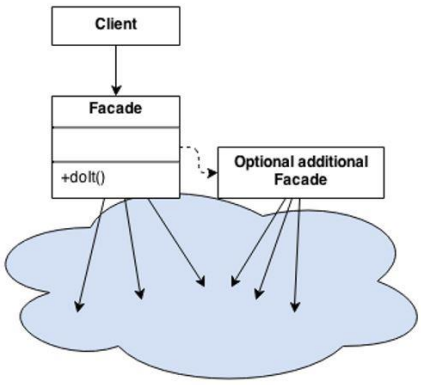
How to go about it?

Facade Pattern

- Structural Pattern
- Provides a unified interface to a set of interfaces in a subsystem.
- Defines a higher-level interface that makes the subsystem easier to use.
- Wraps a complicated subsystem with a simpler interface.
- The principal front of a building, that faces on to a street or open space
- A deceptive outward appearance
- Involves a single class which provides simplified methods required by client and delegates calls to methods of existing system classes.
- Can be used to add an interface to existing system to hide its complexities.



General-building-façade: From Wikimedia Commons



Facade objects are often Singletons because only one Facade object is required.